VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY



NGUYEN THANH TUNG – 523H0192

DUONG TRAN QUOC KHANH – 523H0040

NGUYEN NAM PHONG – 523H0168

TECHHAVEN: AN E-COMMERCE WEBSITE FOR COMPUTERS AND COMPUTER COMPONENTS BUILT WITH NODE.JS

FINAL PROJECT REPORT

WEB PROGRAMMING WITH NODE.JS

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY



NGUYEN THANH TUNG – 523H0192

DUONG TRAN QUOC KHANH – 523H0040

NGUYEN NAM PHONG – 523H0168

TECHHAVEN: AN E-COMMERCE WEBSITE FOR COMPUTERS AND COMPUTER COMPONENTS BUILT WITH NODE.JS

FINAL PROJECT REPORT

WEB PROGRAMMING WITH NODE.JS

Advised by

Lecturer Duong Huu Phuoc

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our lecturer, Duong Huu Phuoc, for the guidance given throughout the final project of the Web Programming with Node.js course. The lectures and the detailed project description showed us how to turn a list of requirements into a working, deployable web application, and the emphasis on quality pushed us to care about validation, concurrency and deployment rather than only about pages that look right.

We would also like to thank the Faculty of Information Technology, Ton Duc Thang University, for providing a supportive learning environment and equipping us with the foundational knowledge necessary to complete this project.

Due to limitations in time and knowledge, this report may inevitably contain certain shortcomings. We sincerely welcome any comments and suggestions from our lecturer to further improve and refine our work.

Thank you sincerely.

Ho Chi Minh City, November 2026

Authors

(Signature and full name)

Nguyen Thanh Tung	Duong Tran Quoc Khanh	Nguyen Nam Phong

DECLARATION OF AUTHORSHIP

We hereby declare that this report and the accompanying source code are our own work, prepared under the guidance of our lecturer, Duong Huu Phuoc. The system design, implementation, test results and screenshots presented in this report were produced by our group, and all test results were obtained by actually running the tests against the implemented system. The demo data were generated by our seed script. All external sources are cited in the references.

In the event of any violation or misconduct, we accept full responsibility for the content of this report. Ton Duc Thang University shall not be held responsible for any intellectual property or copyright infringement committed by the team during the preparation of this report, if any.

Ho Chi Minh City, November 2026

Authors

(Signature and full name)

Nguyen Thanh Tung	Duong Tran Quoc Khanh	Nguyen Nam Phong

TABLE OF CONTENTS

LIST OF FIGURES vii
LIST OF TABLES x
LIST OF ABBREVIATIONS xi
ABSTRACT xii
CHAPTER 1. INTRODUCTION 1
1.1 Background and Objectives 1
1.2 Scope and Group Organization 1
CHAPTER 2. SYSTEM OVERVIEW 3
CHAPTER 3. REQUIREMENTS ANALYSIS 5
3.1 Actors and Use Cases 5
3.2 Business Rules 5
3.3 Non-Functional Requirements 6
CHAPTER 4. SYSTEM ARCHITECTURE AND DESIGN 8
4.1 Overall Architecture 8
4.2 Asynchronous Communication and Stateless Design 9
4.3 Database Design 10
4.4 API Design 12
4.5 Main Processing Flows 12
CHAPTER 5. IMPLEMENTATION 14
5.1 Accounts and Authentication 14
5.2 Catalogue, Search and Product Images 14
5.3 Cart, Checkout and Loyalty 15
5.4 Real-Time Reviews and Notifications 16
5.5 Administration and Dashboards 16
5.6 Storefront Features Inspired by Live Stores 16
5.7 AI Features 17
5.8 Security Measures 18
CHAPTER 6. DEPLOYMENT, SCALING AND BONUS FEATURES 20
6.1 Docker Compose Deployment 20
6.2 Horizontal Scaling 20
6.3 Bonus Features 21
CHAPTER 7. TESTING 22
7.1 Test Plan 22
7.2 Results 22
CHAPTER 8. REMAINING LIMITATIONS AND IMPROVEMENTS 24

CHAPTER 9. CONCLUSION **25**
INDIVIDUAL CONTRIBUTION SUMMARY **26**
APPENDIX A. SYSTEM DEMONSTRATION **27**
A.1 Interface Design 27
A.2 Landing Page and Catalogue 27
A.3 Product Details and Reviews 31
A.4 Cart and Checkout 33
A.5 Accounts, Orders and Shopping Aids 38
A.6 Administrator Console 47
A.7 AI Assistant, E-mail, Responsive Design and Infrastructure 55
APPENDIX B. COMPLETE TEST RESULTS **63**
APPENDIX C. REQUIREMENTS TRACEABILITY AND DEMO ACCOUNTS **66**
REFERENCES **68**

LIST OF FIGURES

Figure 3.1. Use-case diagram of TechHaven 5
Figure 4.1. System architecture 8
Figure 4.2. Asynchronous channels between services 9
Figure 4.3. Entity-relationship diagram 11
Figure 4.4. Checkout sequence 13
Figure 4.5. Order status state machine 13
Figure 6.1. Docker Compose deployment 20
Figure A.1. Landing page 28
Figure A.2. Shop by need and Hot Deals with page numbers 28
Figure A.3. Category section with brand tabs and pagination 29
Figure A.4. Catalogue in grid view 29
Figure A.5. Laptops filtered by brand and price, sorted by price, list view 30
Figure A.6. Products on sale sorted by the biggest discount 30
Figure A.7. ElasticSearch suggestions while typing 31
Figure A.8. Product page with a 10-image gallery and variants 31
Figure A.9. Full-screen image viewer 32
Figure A.10. Original price, discount badge and five-line overview 32
Figure A.11. Ratings, AI sentiment summary and labelled comments 33
Figure A.12. A comment from another browser appears instantly (WebSocket) 33
Figure A.13. Mini-cart after "Add to cart" 34
Figure A.14. Cart with a valid discount code 34
Figure A.15. Checkout step 1 - shipping (guest) 35
Figure A.16. Checkout step 2 - payment 35
Figure A.17. Checkout step 3 - review and confirm 36
Figure A.18. Saved addresses with the default pre-selected 36
Figure A.19. Spending loyalty points 37
Figure A.20. Order success screen after a guest checkout 38

Figure A.21. Registration with e-mail, full name and address 39
Figure A.22. Sign-in page 39
Figure A.23. Password recovery 40
Figure A.24. Profile, password and addresses 41
Figure A.25. Order history 42
Figure A.26. Order details with status history 43
Figure A.27. Tracking an order without signing in 44
Figure A.28. Products selected for comparison 45
Figure A.29. Comparison of three laptops 46
Figure A.30. Wish list 47
Figure A.31. Image manager with 4 of 10 images 48
Figure A.32. Product form with 10 of 10 images; the 11th file is skipped 49
Figure A.33. Variants with selling price, original price, cost and stock 49
Figure A.34. Product management 50
Figure A.35. Category management 50
Figure A.36. Simple dashboard 51
Figure A.37. Actionable insights on the simple dashboard 51
Figure A.38. Advanced dashboard - yearly view (default) 52
Figure A.39. Advanced dashboard - custom range grouped by week 53
Figure A.40. Order list, newest first, 20 per page 53
Figure A.41. Order details and status update 54
Figure A.42. User management 54
Figure A.43. Discount codes with usage and orders 55
Figure A.44. AI assistant (Gemini): gaming laptops under 30 million VND and a follow-up question (Vietnamese) 56
Figure A.45. AI assistant (Gemini): a specific graphics card (English) 56
Figure A.46. Mailpit inbox with the e-mails sent by the shop. 57
Figure A.47. Order confirmation e-mail 57
Figure A.48. Landing page on a phone 58
Figure A.49. Product page on a phone 59

Figure A.50. Cart on a phone 60
Figure A.51. Catalogue on a tablet.....61
Figure A.52. Durable queues in RabbitMQ 62

LIST OF TABLES

Table 2.1. Technologies used 4
Table 3.1. Business rules 6
Table 4.1. Services 8
Table 4.2. Asynchronous messages 9
Table 4.3. Main tables and constraints 11
Table 4.4. Main REST endpoints (prefix /api) 12
Table 5.1. Features observed on live stores and their implementation 17
Table 5.2. Security measures 18
Table 6.1. Distribution of 100 concurrent requests over 5 API replicas 21
Table 7.1. Test results by category 22
Table 8.1. Remaining limitations 24
Table 9.1. Individual contribution summary 26
Table B.1. Complete test results 63
Table C.1. Rubric items, implementation and verification 66
Table C.2. Demo accounts and test data 67

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
COD	Cash on Delivery
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
ERD	Entity-Relationship Diagram
JWT	JSON Web Token
LLM	Large Language Model
OAuth	Open Authorization
REST	Representational State Transfer
SEO	Search Engine Optimisation
SKU	Stock Keeping Unit
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
SQL	Structured Query Language
UI / UX	User Interface / User Experience
VAT	Value-Added Tax
VND (₫)	Vietnamese Dong
WS	WebSocket
E## / U##	End-to-end / unit test identifiers used in this report

ABSTRACT

This report documents the design, implementation and testing of TechHaven, an e-commerce website that sells only computers and computer components. The back end is written in JavaScript on Node.js with Express 5, the front end is a React single-page application with Bootstrap 5, and PostgreSQL stores the business data. The site implements every customer feature of the assignment - a landing page with New Products, Best Sellers and category sections, a paginated catalogue with sorting, filtering and keyword search, product pages with variants that each track their own stock, comments and star ratings updated in real time over WebSockets, a cart that updates without reloading, a three-step checkout that also works for guests, 5-character discount codes limited to 10 uses, a 10% loyalty programme, order tracking and Google sign-in - and an administration area with a simple and an advanced dashboard and management of products, categories, users, orders and discount codes. Administrators can attach from 1 to 10 illustration images to each product by upload, drag and drop or https link, reorder them and choose the cover. After studying live Vietnamese computer stores, the storefront also offers promotional prices with discount badges, key-specification chips, a wish list, product comparison, recently viewed products, "Buy now", a mini-cart, order tracking without an account, an image viewer, and per-page SEO metadata with a sitemap.

The system follows a microservices architecture designed for horizontal scaling: an Nginx gateway load-balances three stateless API replicas and two realtime replicas, while the mailer, search and assistant services are decoupled through RabbitMQ and Redis publish/subscribe. All four bonus features are implemented: a GitHub Actions CI/CD pipeline, the microservices architecture, ElasticSearch product search, and AI features - a shopping-assistant chatbot that recommends products from the live catalogue in Vietnamese or English with replies written by Google Gemini, and sentiment analysis of product comments. All 15 containers start with docker compose up -d, and every e-mail is delivered to a built-in Mailpit inbox

unless Gmail is configured. Twenty-five unit tests, a 66-step end-to-end scenario (including the delivery of the confirmation e-mail), automated browser walkthroughs on desktop, tablet and phone viewports, and scaling and fault-injection experiments all passed. Google sign-in is implemented but requires the deployer's OAuth credentials and could not be tested end to end.

Keywords: Node.js, Express, React, PostgreSQL, e-commerce, microservices, RabbitMQ, ElasticSearch, WebSocket, sentiment analysis, chatbot, Docker Compose, horizontal scaling.

CHAPTER 1. INTRODUCTION

1.1 Background and Objectives

Computers and computer components are expensive, come in many configurations and are compared carefully before purchase, so an online store for them must present variants with their own price and stock, help customers narrow down a large catalogue, show trustworthy reviews and make checkout fast - also for visitors who do not want an account. The shop owner, in turn, needs to process orders, control inventory, run promotions and understand sales over time, and the system must cope with traffic peaks by adding servers.

The objective of this project is to build and deploy such a store, **TechHaven**, as required by the final project of the Web Programming with Node.js course (502070): a back end on a Node.js framework, any front-end framework, every customer and administrator feature of the project description, a responsive interface, a design for horizontal scaling, and deployment with Docker Compose so that the whole system starts with `docker compose up -d`. The group also targeted all four bonus features: a CI/CD pipeline, a microservices architecture with asynchronous communication, AI-related features, and ElasticSearch for product search. Finally, the storefront was compared with live Vietnamese computer stores (GEARVN, Phong Vu) and the shopping aids customers expect there were added, without adding any administrator feature beyond those the description allows.

1.2 Scope and Group Organization

The scope covers the customer store, the administration area, the supporting services and their deployment. As required, there are two user types: customers and a single administrator. Payment is simulated (cash on delivery, bank transfer and a demo card gateway that never stores card data); every e-mail goes to a built-in Mailpit inbox (http://localhost:8025), or through Gmail SMTP when an account is configured; Google sign-in requires an OAuth client created by the deployer; the AI assistant uses Google Gemini through a free-tier API key supplied by the deployer,

and answers with its own rules when no key is configured; the demo product images are illustrations generated by the application, and administrators can replace them with real photos. The work was organised in modules - API and database, customer pages, administration pages, supporting services and DevOps, and testing; the contribution of each member is summarised in the Individual Contribution Summary.

CHAPTER 2. SYSTEM OVERVIEW

TechHaven is a single-page web application backed by a set of Node.js services. Visitors can browse a landing page with **New Products**, **Best Sellers** and five category sections, open a dedicated catalogue with pagination, eight sort orders and filters by brand, price range, rating, stock and category, and search with typo tolerance and suggestions. A product page shows the gallery (1 to 10 images), the variants with their own price and stock, the specifications, and a review section where anyone can comment and signed-in users can rate with stars; new comments and ratings appear on every open page without reloading.

The cart is re-priced by the server on every change and shows the subtotal, the discount, 10% VAT, the shipping fee and the loyalty points to be earned. Checkout has three steps (shipping, payment, review). A guest who checks out gets an account automatically and receives a link to set a password; a signed-in customer has the default address pre-filled, can choose another saved address and can spend loyalty points. After the order the customer sees a success screen and receives a confirmation e-mail, and can later follow the status history of every order.

The administrator works in a separate area with a **simple dashboard** (users, new users, orders, revenue, best sellers) and an **advanced dashboard** (yearly by default, or quarterly, monthly, weekly or a custom range, with orders, revenue, profit, products and product types sold). The administrator manages products - variants, inventory and **from 1 to 10 illustration images per product** - categories, users (view, edit, ban), orders (newest first, 20 per page, period filters, status changes) and discount codes, and nothing else, as the project description requires.

Shopping aids common on live stores complete the experience: promotional prices with the original price struck through and a "-x%" badge, a Hot Deals section, key-specification chips on product cards, brand pills and tags, a wish list, comparison of up to four products, recently viewed products, "Buy now", a mini-cart drawer,

order tracking with the order number and e-mail, an image viewer, floating hotline and back-to-top buttons, and an **AI shopping assistant**. Every product list - including the landing-page sections and related products - has page numbers. Table 2.1 lists the technologies used.

Table 2.1. *Technologies used*

Layer	Technology	Purpose
Front end	React 18, React Router, Vite, Bootstrap 5, Chart.js, Socket.IO client	Single-page store and admin dashboard, responsive layout, charts, live updates
Back end	Node.js 22, Express 5, pg, bcryptjs, jsonwebtoken, multer	REST API, authentication, validation, image upload
Database	PostgreSQL 16	Business data, transactions, row locks, uploaded images
Messaging	RabbitMQ 3.13, Redis 7 publish/subscribe	Asynchronous e-mail jobs and domain events; live notifications; rate limits
Search	ElasticSearch 8.15	Full-text product search, typo tolerance, suggestions
AI	sentiment (AFINN-165) + Vietnamese lexicon, rule-based intent parser, Google Gemini API (@google/genai, gemini-3.5-flash-lite)	Review sentiment analysis; shopping-assistant chatbot
Realtime / mail	Socket.IO 4, Nodemailer, Mailpit	WebSocket push; SMTP delivery to the Mailpit inbox or Gmail
Deployment	Docker, Docker Compose, Nginx 1.27	Containers, reverse proxy and load balancer
Quality	node --test, end-to-end script, Puppeteer, GitHub Actions	Unit, API and UI tests; continuous integration

CHAPTER 3. REQUIREMENTS ANALYSIS

3.1 Actors and Use Cases

A **Guest** is any visitor who has not signed in and can browse, search, comment, use the cart and check out. A **Customer** is signed in, inherits every guest use case, and can also rate products, manage the profile and addresses, follow orders and spend loyalty points. The single **Administrator** manages the shop (Figure 3.1).

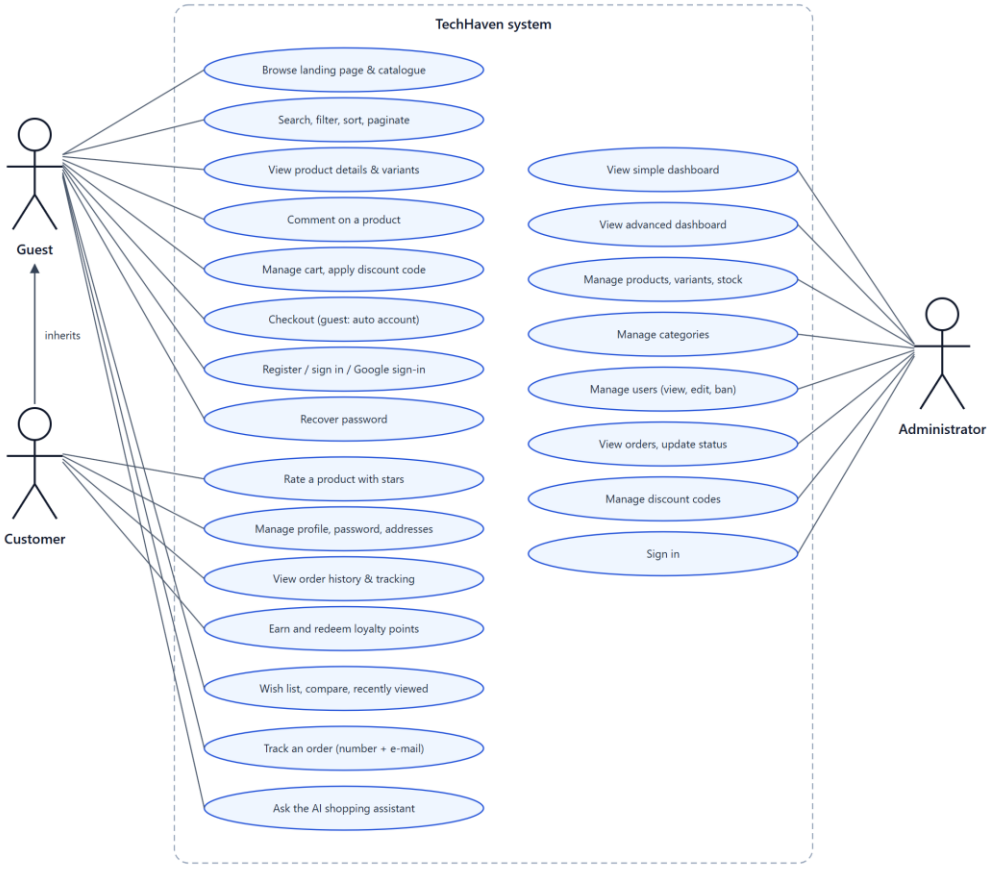


Figure 3.1. *Use-case diagram of TechHaven*

3.2 Business Rules

The project description fixes several rules precisely. Table 3.1 states them as implemented; the pricing rules live in one module (`services/api/src/lib/pricing.js`) shared by the cart preview and the checkout, so the two can never disagree.

Table 3.1. *Business rules*

Rule	Implementation
Registration	Only e-mail, full name and shipping address are entered. The password is chosen afterwards through a one-time link sent by e-mail (valid 72 hours).
Guest checkout	If the e-mail has no account, one is created during checkout with the address and a set-password link is e-mailed; the order appears in the history after sign-in.
Users	Customers and exactly one administrator (partial unique index on users.role).
Variants	Every demo product has at least two variants; each has its own SKU, price, cost and stock.
Product images	From 1 to 10 illustration images per product (PNG, JPG, WEBP or GIF up to 3 MB, or an https link); the first image is the cover.
Product page	Name, price, brand, variants, at least three images and an overview of at least five lines shown next to the price; comments and stars below.
Pagination	Every product list shows page numbers, even with a single page: catalogue, search, landing-page sections, related and recently viewed products, wish list, admin list.
Promotions	A variant may have an original price, which must be higher than the selling price; the discount percentage is computed from both.
Discount codes	Exactly 5 letters or digits, created by the administrator, no expiry, usage limit 1 to 10; effect shown before payment.
Tax and shipping	VAT 10% on the subtotal after discount; shipping 30,000₫, free from 5,000,000₫.
Loyalty points	Earned points = floor(10% × total / 1,000); 1 point = 1,000₫; usable immediately on the next order.
Reviews	Anyone may comment; star ratings require sign-in but no purchase; one rating per user and product.
Orders	pending → confirmed → shipping → delivered; pending or confirmed → cancelled; every change is time-stamped.

3.3 Non-Functional Requirements

- **Deployment:** the whole system starts with `docker compose up -d`; dependencies are installed inside the images.
- **Horizontal scaling:** stateless services behind a load balancer; adding instances needs no code change.
- **Responsiveness and usability:** layouts for phone, tablet and desktop; formatted prices (75,299,000₫) and dates; no full-page reloads for the cart, comments and ratings.
- **Load time:** the administration pages and Chart.js are split into separate files that load only when the administrator opens them, so a shopper downloads about 96 KB of compressed JavaScript instead of 173 KB; static files are compressed by the gateway and cached for one year.

- **Security:** hashed passwords, httpOnly cookies, parameterised SQL, server-side validation, rate limiting, file-signature checks on uploads.
- **SEO and navigation:** categories and tags, per-page titles and meta descriptions, Product structured data (JSON-LD), robots.txt and sitemap.xml.
- **Reliability:** checkout in one database transaction; e-mail and indexing are asynchronous and retried.

CHAPTER 4. SYSTEM ARCHITECTURE AND DESIGN

4.1 Overall Architecture

The browser talks to one entry point, the **gateway** (Nginx). Requests under `/api/` are spread round-robin over the API replicas, WebSocket connections under `/socket.io/` over the realtime replicas, and all other paths are served by the **frontend** container (Figure 4.1). The gateway re-resolves service names through Docker DNS every five seconds, so replicas added with `docker compose up -d --scale api=5` join the rotation automatically, and it retries another replica if one does not answer within two seconds.

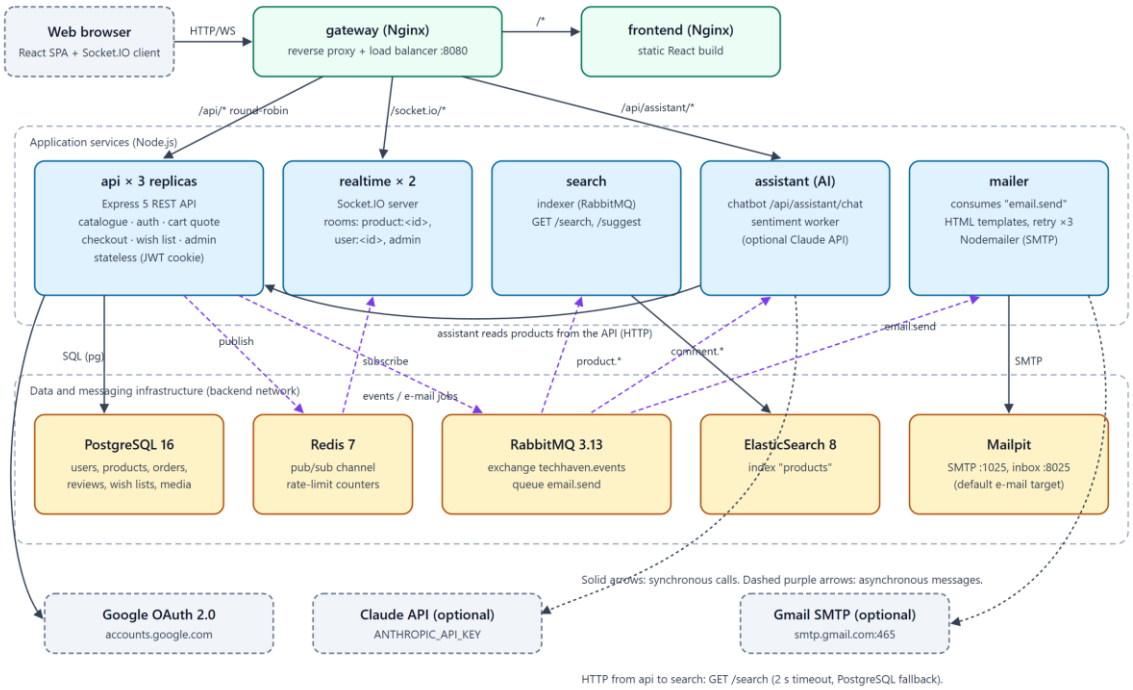


Figure 4.1. *System architecture*

Table 4.1. *Services*

Service	Responsibility	Instances
gateway	Public entry point; reverse proxy, load balancing, WebSocket upgrade	1
frontend	React store and admin dashboard served by Nginx	1
api	Catalogue, accounts, cart pricing, checkout, loyalty, reviews, administration, image upload; schema migration and seeding	3 (scalable)
realtime	Socket.IO connections; relays events from Redis to rooms	2 (scalable)

Service	Responsibility	Instances
mailer	Consumes the e-mail queue; renders templates; sends through SMTP	1
search	Keeps the ElasticSearch index in sync; search and suggest API	1
assistant	AI: shopping-assistant chatbot and sentiment analysis of comments	1
postgres, redis, rabbitmq, elasticsearch, mailpit	Data, pub/sub and rate limits, message broker, search index, e-mail inbox	1 each

4.2 Asynchronous Communication and Stateless Design

Only two interactions are synchronous: browser to API, and API to the search service for keyword queries (2-second timeout with a PostgreSQL fallback). Everything else travels through RabbitMQ and Redis (Figure 4.2, Table 4.2), so a slow mail server or a restarting search service never blocks a customer.

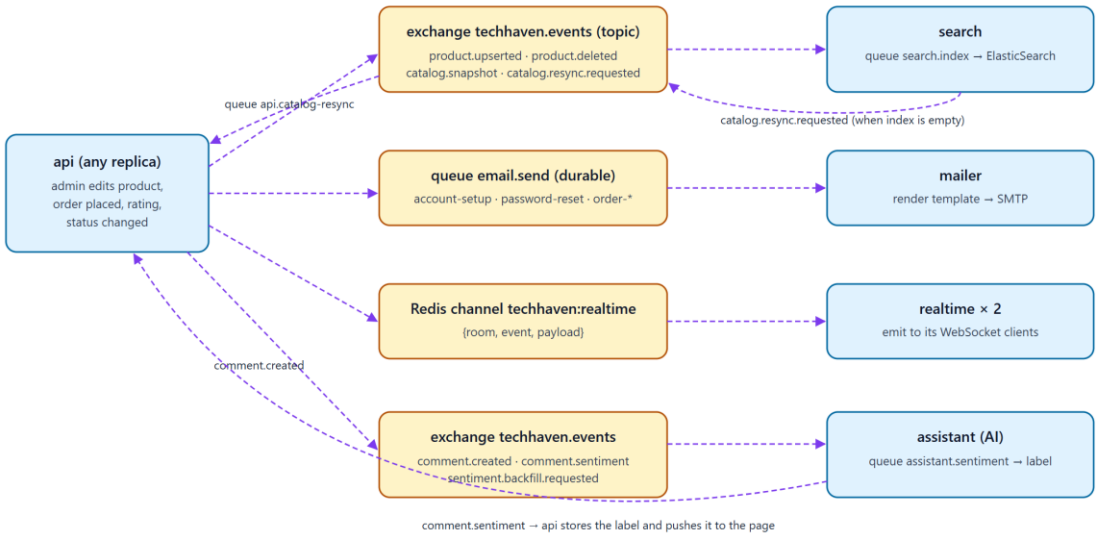


Figure 4.2. *Asynchronous channels between services*

Table 4.2. *Asynchronous messages*

Channel	Producer → consumer	Purpose
Queue email.send (durable)	api → mailer	Account setup, password reset, order confirmation and status e-mails; retried up to 3 times.
Exchange techhaven.events: product.upserted / product.deleted	api → search	Keep the index in sync after product edits (including images), sales and ratings.

Channel	Producer → consumer	Purpose
catalog.snapshot / catalog.resync.requested	api ↔ search	Full re-index when the index is empty; the shared queue makes exactly one API replica answer.
comment.created / comment.sentiment	api → assistant → api	Every new comment is classified; the label is stored and pushed to the product page. sentiment.backfill.requested re-sends unclassified comments.
Redis channel techhaven:realtime	api → every realtime replica	comment:new, comment:sentiment, rating:updated, order:status, order:new pushed to WebSocket rooms.

Any API replica can serve any request: sessions are signed JWT cookies; carts live in the browser and are re-priced by the server; uploaded images are stored in PostgreSQL rather than on a container disk; rate limits are counted in Redis; start-up seeding is serialised with a PostgreSQL advisory lock; and WebSockets use the WebSocket transport only, so no sticky sessions are needed.

4.3 Database Design

Figure 4.3 shows the 15 tables of `services/api/src/schema.sql`. Order lines copy the product name, price and cost, so order history and profit stay correct after a product is edited or deleted. Product images are rows of `product_images` (ordered by position; the first is the cover) that point either to an uploaded file in `media` or to an https link.



Figure 4.3. *Entity-relationship diagram*

Table 4.3. *Main tables and constraints*

Table	Key columns and constraints
users	email and google_id unique; password_hash nullable (guest or Google accounts); one administrator; loyalty_points ≥ 0
addresses	one default address per user (partial unique index); up to 10 per user
products / variants	slug and sku unique; price > 0; stock ≥ 0; optional compare_at_price (original price); read-model columns min_price, min_compare_price, sold_count, avg_rating
product_images / media	1 to 10 rows per product, ordered by position; media stores uploaded bytes and the detected MIME type
discount_codes	code matches ^[A-Z0-9]{5}$; usage_limit 1-10; used_count ≤ usage_limit
orders / order_items	status limited to five values; snapshot of contact, address, prices, cost, discount and points
order_status_history	every status change with time, note and administrator
comments / ratings	guests may comment; sentiment label and score set by the assistant; one rating (1-5) per user and product
wishlists	(user_id, product_id) primary key; wish lists of signed-in customers

4.4 API Design

Table 4.4. *Main REST endpoints (prefix /api)*

Group	Endpoints	Access
Authentication	POST /auth/register, /auth/login, /auth/logout, /auth/forgot-password, /auth/set-password, /auth/change-password; GET /auth/me, /auth/google, /auth/google/callback	Public
Profile	GET, PUT /profile; GET, POST /profile/addresses; PUT, DELETE /profile/addresses/:id; POST /profile/addresses/:id/default	Customer
Catalogue	GET /home, /categories, /brands, /tags, /products (incl. onSale, sort=discount), /products/:slug, /products/by-slugs, /search/suggest, /config	Public
Wish list	GET /wishlist; PUT, DELETE /wishlist/:productId; POST /wishlist/merge	Customer
AI assistant	POST /assistant/chat (assistant service, routed by the gateway)	Public
SEO	/robots.txt, /sitemap.xml (gateway → GET /seo/robots.txt, /seo/sitemap.xml)	Public
Reviews	GET, POST /products/:id/comments; PUT /products/:id/rating	Public / rating: signed in
Cart, orders	POST /cart/quote, /orders, /orders/lookup (number + e-mail); GET /orders, /orders/:number	Public / history: signed in
Admin catalogue	GET, POST /admin/products; GET, PUT, DELETE /admin/products/:id; POST /admin/uploads (1-10 images); PATCH /admin/variants/:id/stock; categories CRUD	Administrator
Admin other	GET /admin/dashboard/summary, /advanced; users, orders (PATCH status), discounts	Administrator

4.5 Main Processing Flows

Checkout (Figure 4.4). While the cart changes, the browser calls `POST /cart/quote`. When the order is placed, the API repeats the calculation inside one transaction after locking the variants, the discount code and the buyer with `SELECT ... FOR UPDATE`; the code usage is incremented only while `used_count < usage_limit`, so the last unit or the last use can never be sold twice. E-mail and re-indexing happen after the commit through RabbitMQ.

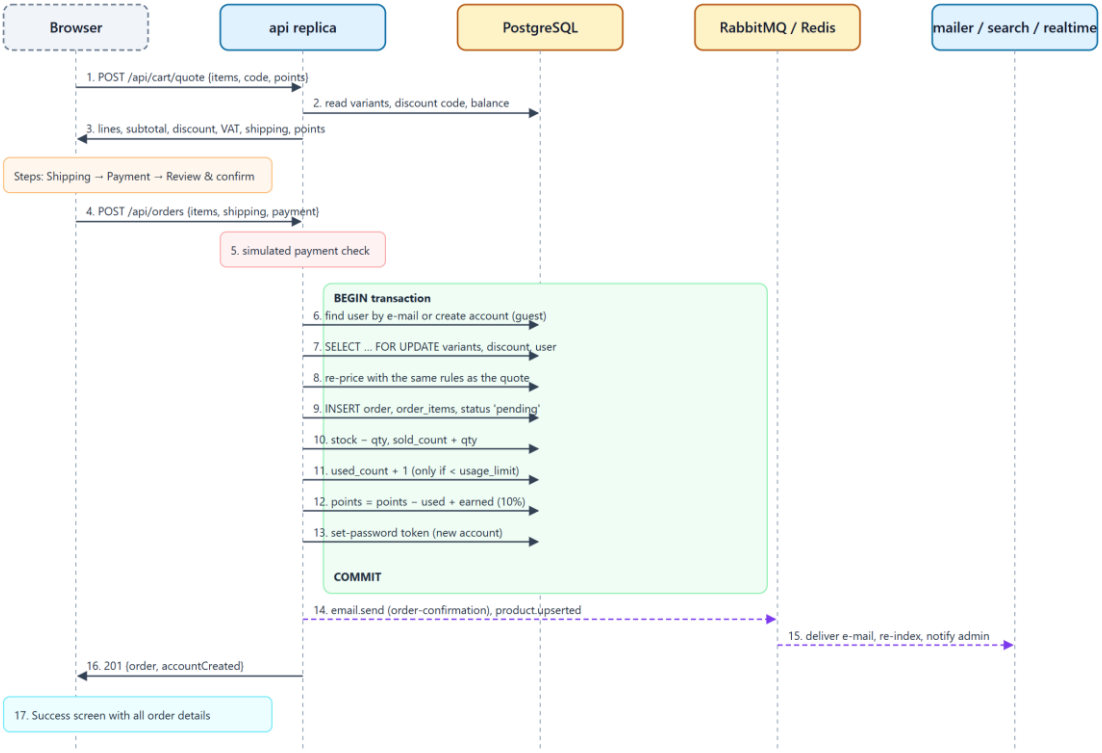


Figure 4.4. *Checkout sequence*

Order status (Figure 4.5). The API accepts only the transitions of the state machine; each change is stored in `order_status_history`, e-mailed and pushed to the customer's open pages. Cancelling restores stock, refunds the points used, removes the points earned and releases the discount use.

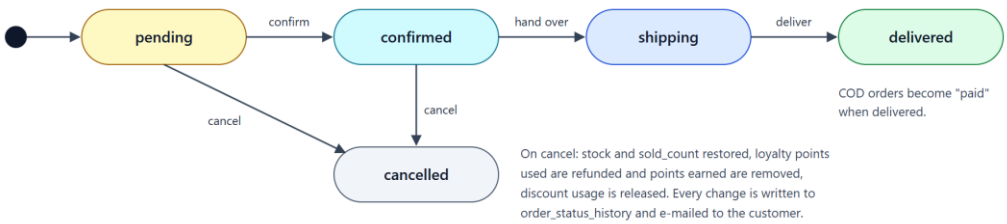


Figure 4.5. *Order status state machine*

CHAPTER 5. IMPLEMENTATION

The source is organised as one folder per service (`frontend`, `gateway`, `services/api`, `services/realtime`, `services/mailer`, `services/search`, `services/assistant`) plus `tests` and `.github/workflows`. On the first start the API creates the schema and loads deterministic demo data: 9 categories, 118 products (13 or 14 per category, so every category spans two catalogue pages) with 264 variants - two or three per product - and 4 images each (about half of them on promotion), 160 customers, about one thousand orders since January 2024, six discount codes, and ratings and comments in English and Vietnamese. Screens of every feature are shown in Appendix A.

5.1 Accounts and Authentication

Passwords are hashed with bcrypt; after sign-in the API issues a JWT in an httpOnly, SameSite=Lax cookie that every replica can verify. Registration stores only e-mail, name and address and sends a one-time link (stored as a SHA-256 hash, single use) to choose the password; the same mechanism serves password recovery (link valid one hour). Google sign-in uses the OAuth 2.0 authorization-code flow with a signed `state` cookie; a new Google user gets an account, and an existing account with the same verified e-mail is linked. Customers can edit the profile, change the password and manage up to ten addresses with one default.

5.2 Catalogue, Search and Product Images

Product cards and filters are built by `lib/catalog.js` with parameterised SQL and a whitelist of sort orders (newest, best selling, name A-Z and Z-A, price ascending and descending, rating, biggest discount, relevance). Every product list uses the same paginated component with page numbers - the catalogue (12 per page), the landing-page sections, related and recently viewed products and the wish list - so pagination works "everywhere products are listed", as the description requires. Keyword queries go to the search service, which combines an exact multi-field match

with a typo-tolerant match on names and brands (fuzziness `AUTO:5,8`, so model numbers like 4070 are never fuzzed); if the service is unavailable the API falls back to SQL. The product page shows an **overview of at least five lines** (the first lines of the description) next to the price, above the variants.

Product image management. Each product has from 1 to 10 illustration images. In the product form the administrator can drop files on the upload area or browse for them, paste an https link, drag tiles to reorder them, move any image to the cover position and remove images; a counter shows how many of the 10 slots are used, files beyond the limit are skipped with a message, and the upload area is disabled when the product is full (Figure A.31, Figure A.32). The rules are enforced on the server as well (`lib/images.js`):

- `POST /api/admin/uploads` accepts up to 10 files of at most 3 MB per request and detects the real format from the file signature (PNG, JPEG, GIF, WEBP); a file whose content is not an image is rejected even if its name and declared type say otherwise.
- Uploaded bytes are stored in the `media` table with the detected MIME type and served from `/api/media/:id`, so every API replica can serve every image.
- Saving a product requires between 1 and 10 distinct images, each either an uploaded file or an https URL; the order of the list becomes the gallery order and the first image is the cover.
- The product page shows a gallery with previous/next buttons, keyboard arrows, an "n / N" counter, thumbnails and a full-screen viewer (Figure A.8, Figure A.9).

5.3 Cart, Checkout and Loyalty

The cart is kept in the browser so guests can shop without an account, but every change calls `POST /cart/quote`, which returns authoritative prices, stock warnings, the discount status, VAT, shipping and points. Checkout re-prices everything inside a transaction with row locks, creates the order, its lines and the first history entry, decrements stock, increments the best-seller counter and the code

usage, and updates the loyalty balance (points used are subtracted and the 10% earned is added immediately). Guests receive an automatically created account. The confirmation e-mail is queued after the commit and delivered by the mailer to the Mailpit inbox (Figure A.47) or through Gmail.

5.4 Real-Time Reviews and Notifications

A product page joins the Socket.IO room of that product. When a comment or rating is saved, the API publishes the event on Redis; each realtime replica emits it to its own clients in the room, so every viewer sees it without reloading. The same channel delivers the sentiment label of new comments, order status changes to customers and new orders to the administrator.

5.5 Administration and Dashboards

The simple dashboard aggregates customers, new customers, orders, revenue, a 12-month chart, orders by status and the best sellers, and adds **actionable insights**: orders to confirm and to ship, out-of-stock and low-stock variants with links, and discount codes close to their limit. The advanced dashboard groups orders by year (default), quarter, month, week or day over a chosen range and reports orders, revenue, profit, units and product types sold; profit is item revenue minus item cost, discounts and redeemed points. Order management offers 20 orders per page, newest first, with period filters, and status changes limited to the state machine. User management allows viewing, editing and banning customers; discount management shows each code's usage and the orders that used it. No administrator feature beyond those listed in the description was added.

5.6 Storefront Features Inspired by Live Stores

Two large Vietnamese computer stores, GEARVN and Phong Vu, were studied in November 2026. Table 5.1 lists what customers find there and how TechHaven implements it. All additions are on the customer side; none changes the

rules highlighted in the description (registration fields, guest checkout, discount codes, administrator scope).

Table 5.1. *Features observed on live stores and their implementation*

Feature on live stores	TechHaven implementation
Original price struck through, "-x%" badge, deal section with the biggest discounts	Optional original price per variant (admin form); badge on cards and product page; Hot Deals section; "On sale" filter and "Biggest discount" sort
Key specifications on product cards (CPU, RAM, GPU, screen)	Up to three specification chips chosen by priority from the product specs
Brand logos / brand tabs inside category sections	Brand pills on catalogue pages and the home page; brand tabs in each landing-page category section
Shopping by need (students, gaming, design)	"Shop by need" tiles linked to product tags
Wish list, compare products	Heart and compare buttons on cards; wish list stored in the account (or the browser for guests, merged at sign-in); comparison of up to four products with "differences only"
Recently viewed products	Kept in the browser and shown on the home and product pages
"Buy now" next to "Add to cart"; cart preview	Buy now goes straight to checkout; a mini-cart drawer opens after "Add to cart"
Order tracking from the header	"Track order" page with order number and e-mail, no account needed
Zoomable product gallery	Full-screen viewer with keyboard navigation
Floating hotline / chat buttons, back to top	Floating hotline, AI assistant and back-to-top buttons
Banners with promotions	Rotating hero slides linking to deals and needs

5.7 AI Features

The **assistant** microservice provides the AI bonus features (Figure A.44, Figure A.45, Figure A.11):

- **Shopping assistant chatbot.** A rule-based parser (`intent.js`) understands Vietnamese and English questions - budgets such as "dưới 30 triệu", "under 8m" or "tầm 20tr", product types, intended uses (gaming, office, students, design, programming), brands, model numbers such as "RTX 4070" and store-policy questions. The assistant queries the live catalogue through the API and returns only in-stock products that match, as clickable cards, relaxing the filters step by step when nothing matches. Google Gemini then writes the reply text from those products only (`gemini-3.5-flash-lite` through the `@google/genai` SDK, with the key in `GEMINI_API_KEY`; a free-tier key can be created in Google AI Studio).

The model was chosen by measurement: `gemini-3.8-flash` needed 6 to 27 seconds per reply on the free tier, the lite model about 1.5 to 2 seconds with equally correct Vietnamese. Gemini receives only the candidate products with prices already formatted for the language of the customer, the store policies and the last six turns, and is told to reply in plain text in that language and to mention policies only when asked, so it cannot recommend products the shop does not sell. Follow-up questions keep the context: after "Tư vấn laptop gaming dưới 30 triệu", the question "Có mẫu nào rẻ hơn không?" reuses the product type, use and budget of the previous request and sorts by price. Any error - an invalid key, the free-tier quota being used up, more than 25 seconds - falls back to the template reply, so the feature also works without a key; the `engine` field of each response tells which path answered (`gemini` or `rules`). Figures A.44 and A.45 and the final end-to-end run were answered by Gemini.

- **Sentiment analysis of reviews.** Each new comment is published on RabbitMQ; the assistant scores it with the AFINN-165 lexicon for English and a Vietnamese lexicon with negation handling ("không tốt" is negative) and publishes a positive / neutral / negative label, which the API stores and pushes to open product pages. The product page shows the share of positive, neutral and negative comments ("What buyers say"). On start-up the assistant asks for comments that were never analysed, so the seeded comments are classified too.

5.8 Security Measures

Table 5.2. *Security measures*

Threat	Measure
Password theft	bcrypt hashes; 8-72 characters with letters and digits; one-time tokens stored as SHA-256 hashes with expiry.
Session theft, XSS	httpOnly SameSite=Lax cookie; React and e-mail templates escape output; X-Content-Type-Options: nosniff.
CSRF	SameSite cookies and JSON-only endpoints; signed OAuth state.
SQL injection	Parameterised queries; whitelisted sort keys.
Broken access control	requireAuth / requireAdmin guards; customers read only their own orders, addresses and wish list; order lookup needs the order number and the matching e-mail.
Malicious uploads	Images only, detected from file signatures; 3 MB per file; at most 10 per request and per product.
Brute force and spam	Redis rate limits on sign-in, e-mail sending, comments, checkout and order lookup; per-IP limit on the assistant; generic answers on password recovery.

Threat	Measure
AI misuse	The chatbot only receives catalogue data, may only recommend listed products and never asks for personal data; message length is limited.

CHAPTER 6. DEPLOYMENT, SCALING AND BONUS FEATURES

6.1 Docker Compose Deployment

`docker-compose.yml` defines 12 services (15 containers with replicas) on two networks; health checks order the start-up, and only ports 8080 (website), 8025 (e-mail inbox) and 15672 (RabbitMQ management) are published (Figure 6.1). No `npm install` is needed on the host.

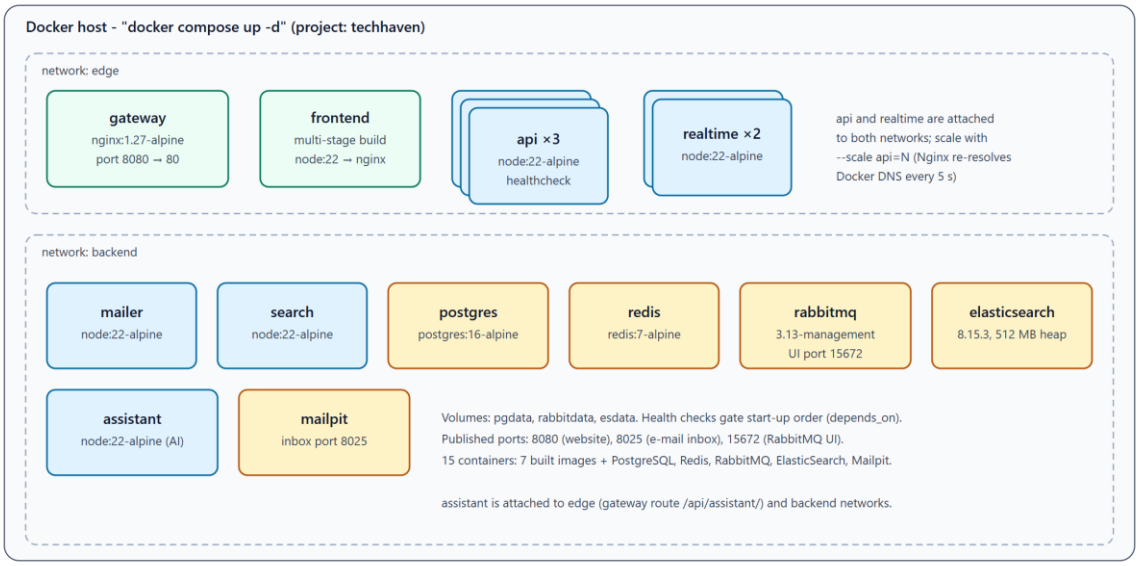


Figure 6.1. *Docker Compose deployment*

6.2 Horizontal Scaling

The API was scaled from three to five replicas while running and 100 concurrent requests were sent through the gateway; each response carries an `X-Instance` header naming the container that served it (Table 6.1). With one replica stopped, 60 requests still succeeded within 2.1 s: the gateway waits at most 2 s (`proxy_connect_timeout 2s`) before retrying another replica (`proxy_next_upstream_tries 3`); before these settings were added the same test took 60 s.

Table 6.1. Distribution of 100 concurrent requests over 5 API replicas

Replica (container id)	Requests served
2aa160b6aa8b	24
2d20883e3a2b	17
2e7aae086a73	18
9f415172cdb3	22
e4ec64cb7174	19
Total (187 ms)	100

6.3 Bonus Features

- **CI/CD (GitHub Actions).** `.github/workflows/ci.yml` runs on every push to `main` and every pull request: unit tests of the API, the mailer and the assistant, checks of the other services and the React build; then it builds the whole stack with `docker compose up -d --build --wait`, waits for the search index, runs the 66-step end-to-end scenario through the gateway and tears the stack down.
- **Microservices with asynchronous communication.** Besides the front end and the data stores there are five Node.js services (api, realtime, mailer, search, assistant) connected through a durable RabbitMQ queue, a topic exchange and Redis publish/subscribe (Section 4.2). Stopping the search service does not break search, stopping the mailer only delays e-mails, and stopping the assistant leaves the shop working.
- **AI features.** Shopping-assistant chatbot and sentiment analysis of comments (Section 5.7).
- **ElasticSearch.** The `products` index uses lower-casing and ASCII folding and a search_as_you_type field; filters, sorting and pagination run inside ElasticSearch, and the index follows product changes through RabbitMQ events.

CHAPTER 7. TESTING

7.1 Test Plan

Four kinds of tests were used. **Unit tests** (`node --test`) cover the pricing, loyalty, discount, validation, image, e-mail template, sentiment and chatbot-parsing rules. The **end-to-end scenario** `tests/e2e-api.mjs` drives the running stack through the gateway with separate guest, customer and administrator sessions and also checks that the confirmation e-mail arrives in the Mailpit inbox. **Automated browser walkthroughs** (Puppeteer with Microsoft Edge) exercise the main journeys on desktop, tablet and phone viewports, including the image manager, the mini-cart, comparison, wish list, order tracking and the AI assistant, and fail on any JavaScript error; they produced the screenshots in Appendix A. **Fault-injection experiments** add replicas and stop containers during traffic.

7.2 Results

The final runs passed 25 of 25 unit tests and 66 of 66 end-to-end checks (Table 7.1); the complete list is in Appendix B. The browser walkthroughs reported no JavaScript errors.

Table 7.1. *Test results by category*

Category	Tests	Pass	Fail	Test IDs
Pricing, discount and loyalty rules	7	7	0	U04-U10
Input validation	5	5	0	U11-U15
Image rules	3	3	0	U01-U03
E-mail templates	3	3	0	U16-U18
Sentiment analysis, chatbot parsing, Gemini prompt and follow-up context	7	7	0	U19-U25
Load balancing	1	1	0	E01
Catalogue, search, product page	8	8	0	E02-E09
Reviews	3	3	0	E10, E11, E13
Accounts and access control	5	5	0	E12, E14-E16, E30
Discount codes	5	5	0	E17-E20, E26
Checkout and loyalty	6	6	0	E21-E25, E27
Orders and status history	6	6	0	E28, E29, E31-E34
Guest checkout and order tracking	4	4	0	E35, E36, E51, E52
Product images (1-10)	7	7	0	E37-E43

Category	Tests	Pass	Fail	Test IDs
Deals, specs, compare, wish list, SEO	8	8	0	E44-E50, E53
AI assistant and sentiment	5	5	0	E54-E58
E-mail delivery (Mailpit)	1	1	0	E59
Dashboards	7	7	0	E60-E66
Total	**89**	**89**	**0**	

The browser walkthroughs found five defects, all corrected before the final run. Two had the same cause: React effects written as arrow expressions returned the value of `window.scrollTo` and `scrollIntoView`, which recent Chromium versions return as a promise, and React then tried to call it as a clean-up function - the second case blanked the whole page after the chatbot answered. Every effect was rewritten with a block body so this cannot recur. The others were an invalid phone `pattern` attribute, a discount code lost when the checkout page was reloaded, and dashboard charts without a fixed height. The fault-injection runs also led to the gateway fail-over settings in Section 6.2.

A final automated layout audit opened 27 pages (store, customer area and administration) at 1366, 768 and 375 pixels wide - 81 page views - and reported any horizontal page scrolling, element sticking out of its card, clipped button text, broken image or JavaScript error; every screenshot was also reviewed by eye. It led to these corrections: on discounted product cards the original price and the discount badge now sit on their own line above the price, because long prices pushed the View button out of the card; on phones the View button is hidden (the whole card is a link), the stars no longer wrap and the card padding is reduced so prices fit at 360 pixels; the order-items table hides the image and unit-price columns on phones; the brand filter shows the ten largest brands with a "+N more" button and becomes one swipeable row on phones; the comparison table keeps its label column visible while scrolling sideways; the nine category tiles fit on one row on wide screens; counts use the singular when needed ("1 item"); and the brand on generated product images moved to the bottom so discount badges no longer cover it. After these changes the audit reported no layout problem and no JavaScript error.

CHAPTER 8. REMAINING LIMITATIONS AND IMPROVEMENTS

The final review compared the implementation with every requirement of the project description (Appendix C). Table 8.1 lists the limitations that remain and the recommended improvement for each.

Table 8.1. *Remaining limitations*

#	Limitation (verified)	Impact	Recommended improvement
1	Google sign-in could not be tested end to end without the deployer's OAuth credentials.	Medium	Create the OAuth client, set the variables in .env and repeat the sign-in scenario before the demonstration.
2	Payment is simulated.	Medium	Integrate a real gateway (e.g. VNPay or Stripe) with refunds for cancelled paid orders.
3	All replicas run on one Docker host; PostgreSQL, Redis, RabbitMQ and ElasticSearch are single instances.	Medium	Deploy on a cluster with replicated or managed data services.
4	The chatbot's own understanding is rule-based; free-form phrasing is handled well only when the Gemini key is configured, and the Gemini free tier has daily request limits.	Low	Use a paid Gemini tier in production, or extend the intent rules with real customer questions.
5	The cart and the recently viewed list are stored in the browser and do not follow a customer to another device.	Low	Synchronise them with the account of signed-in customers.
6	Uploaded images that are removed from a product stay in the media table.	Low	Add a periodic clean-up of media rows no longer referenced by any product.
7	Demo product images are generated illustrations, not photos.	Low	Upload real product photos with the image manager.

CHAPTER 9. CONCLUSION

The group built and deployed TechHaven, an e-commerce website for computers and computer components that implements every customer and administrator feature of the project description, follows its mandatory rules for registration, guest checkout, discount codes, loyalty points and the administrator scope, and starts with a single Docker Compose command. Administrators can attach 1 to 10 illustration images to each product, and the storefront offers the shopping aids of live computer stores. The back end is a set of five Node.js services - a stateless Express API scaled behind Nginx, a Socket.IO realtime service, a mailer, an ElasticSearch-based search service and an AI assistant - connected through RabbitMQ and Redis, and all four bonus features were completed. All 89 automated tests passed. The most useful lessons were to keep every price rule in one tested module, to rely on database transactions and row locks for stock and discount limits, to validate uploads by content rather than by declared type, to move slow work out of the request path, and to ground an AI assistant in the shop's own data.

INDIVIDUAL CONTRIBUTION SUMMARY

To be completed by the team: each member's tasks, repository commits and share of the work. The yellow cells below are placeholders and must be replaced before submission.

Table 9.1. *Individual contribution summary*

Member	Primary responsibility	Summary of contribution
Nguyen Thanh Tung 523H0192	*to be completed*	*to be completed*
Duong Tran Quoc Khanh 523H0040	*to be completed*	*to be completed*
Nguyen Nam Phong 523H0168	*to be completed*	*to be completed*

APPENDIX A. SYSTEM DEMONSTRATION

This appendix walks through the implemented system screen by screen. All screenshots were captured automatically from the running Docker Compose stack with the seeded demo data.

A.1 Interface Design

The store uses a dark navy header with the search box, wish list, cart and account menu, a category bar, white cards on a light background and a sky-blue accent colour; prices are formatted as 13,490,000₫ and statuses are colour-coded. The administrator area has its own sidebar layout.

A.2 Landing Page and Catalogue

The landing page shows rotating slides, category tiles, "Shop by need", Hot Deals, New Products, Best Sellers and five category sections with brand tabs; every section has page numbers (Figure A.1 to Figure A.3). The catalogue offers grid or list view, pagination, nine sort orders, brand pills, tags and filters by brand, price range, rating, stock and promotion (Figure A.4 to Figure A.6); search results come from ElasticSearch and suggestions appear while typing (Figure A.7).

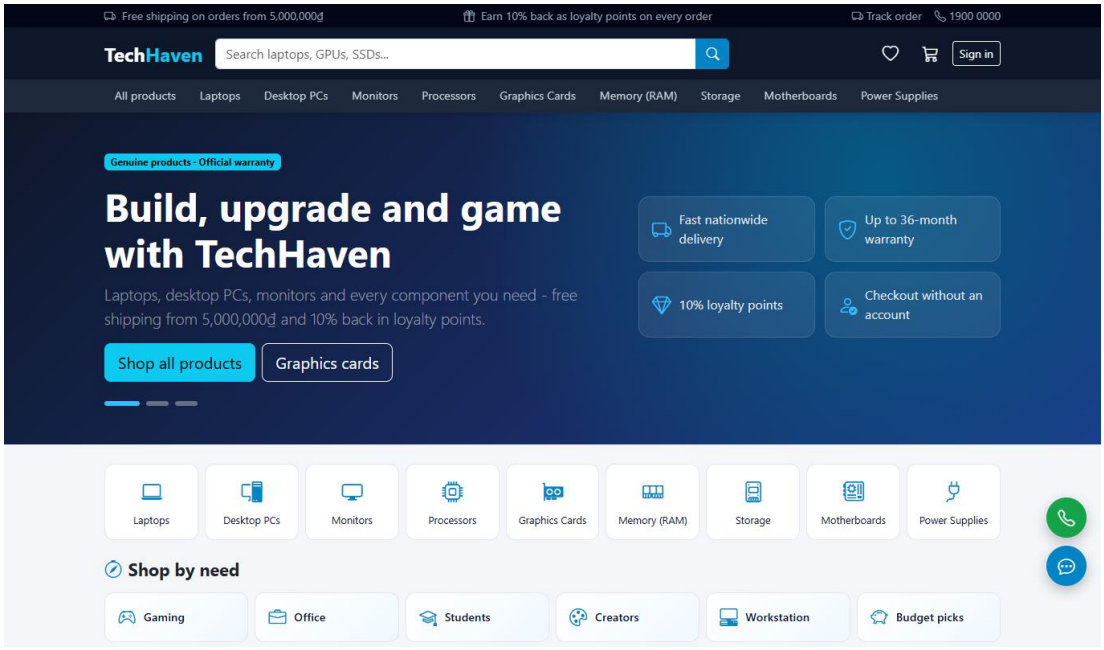


Figure A.1. *Landing page*

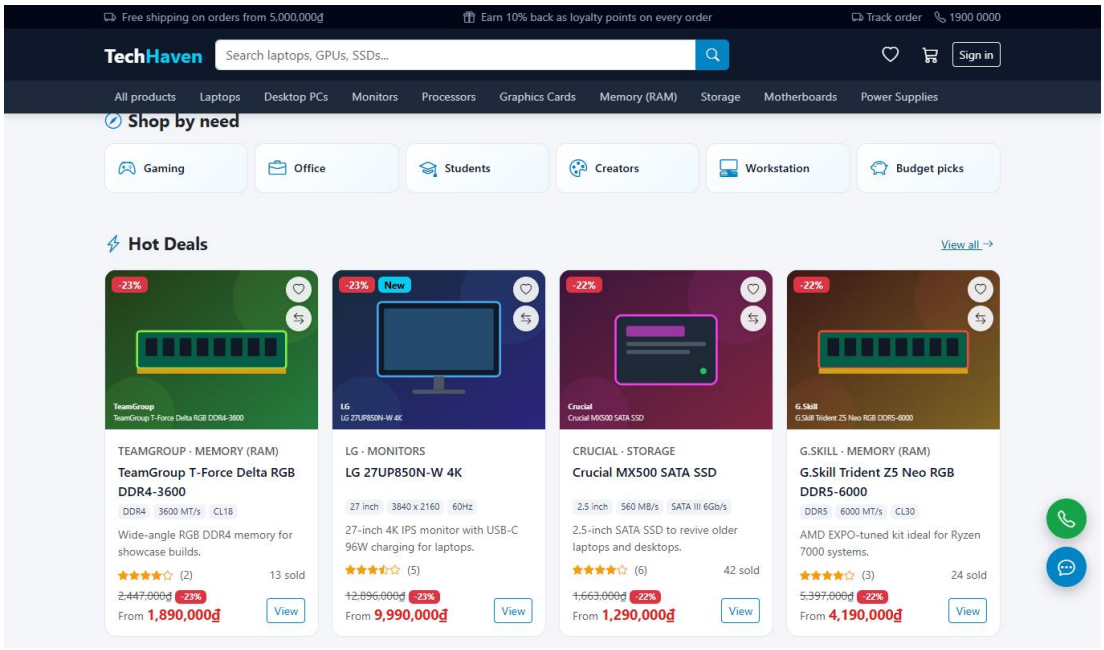


Figure A.2. *Shop by need and Hot Deals with page numbers*

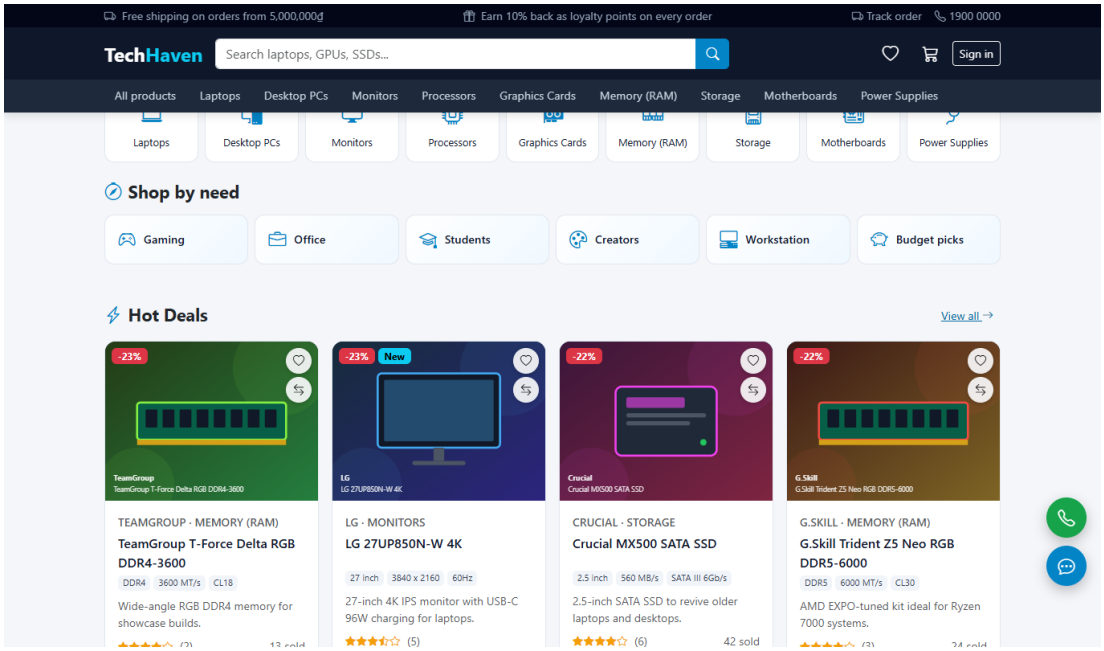


Figure A.3. *Category section with brand tabs and pagination*

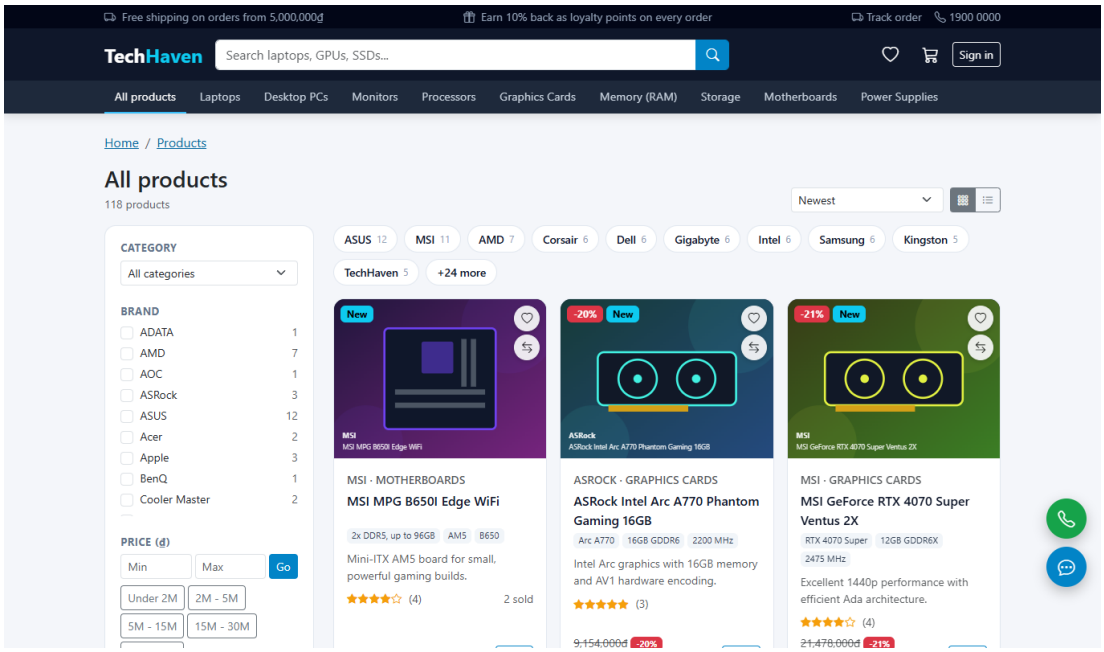


Figure A.4. *Catalogue in grid view*

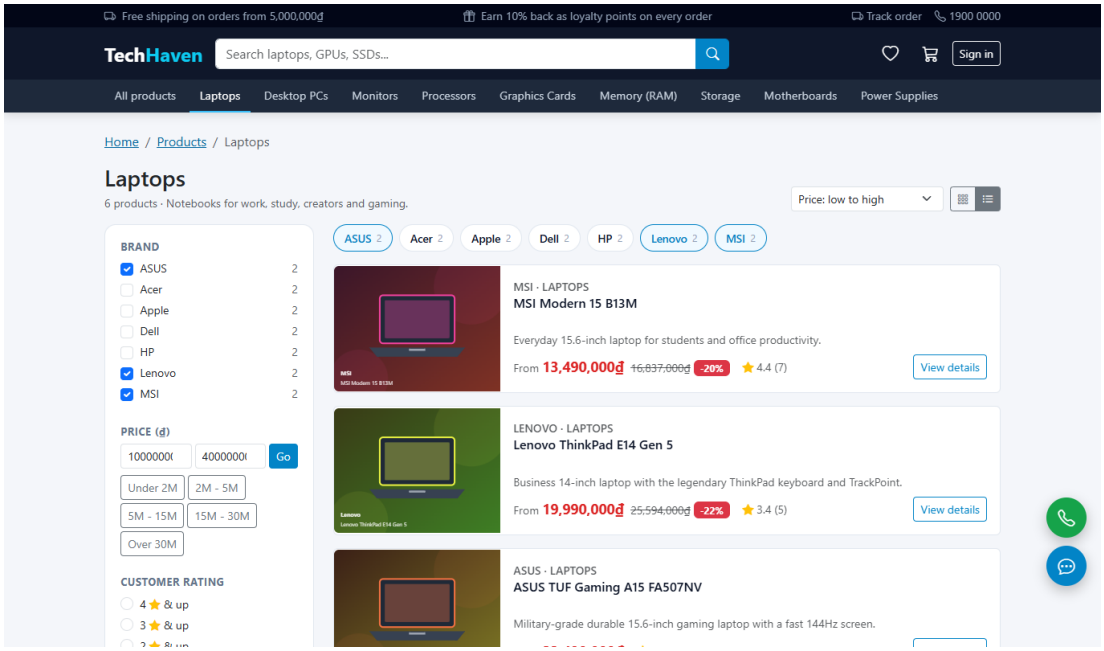


Figure A.5. *Laptops filtered by brand and price, sorted by price, list view*

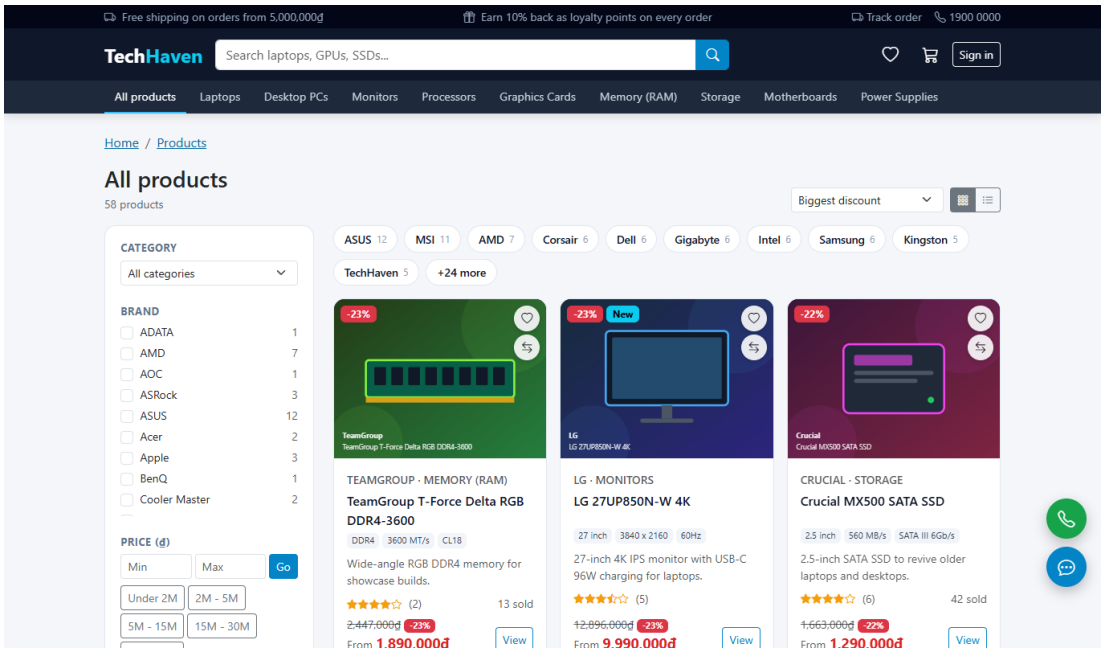


Figure A.6. *Products on sale sorted by the biggest discount*

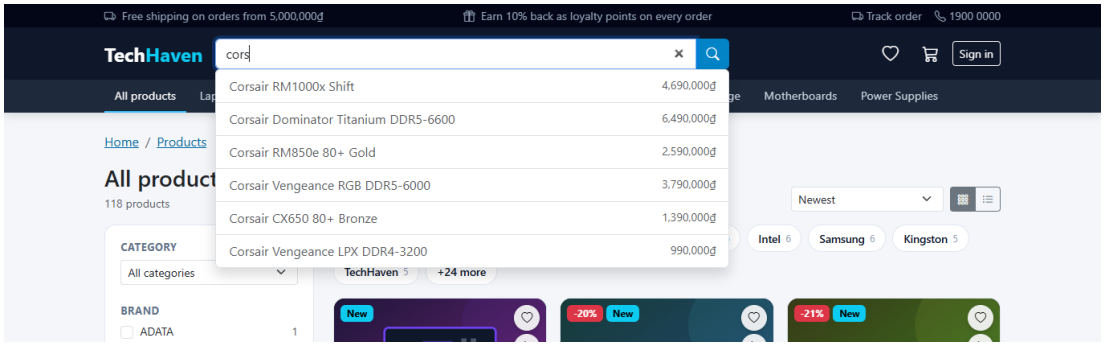


Figure A.7. *ElasticSearch suggestions while typing*

A.3 Product Details and Reviews

The product page shows the gallery (up to ten images, full-screen viewer), the selling and original prices, the overview of at least five lines, the variants with their own price and stock, "Buy now", wish list and compare (Figure A.8 to Figure A.10). Comments and star ratings appear in real time and each comment receives an AI sentiment label; the rating panel summarises what buyers say (Figure A.11, Figure A.12).

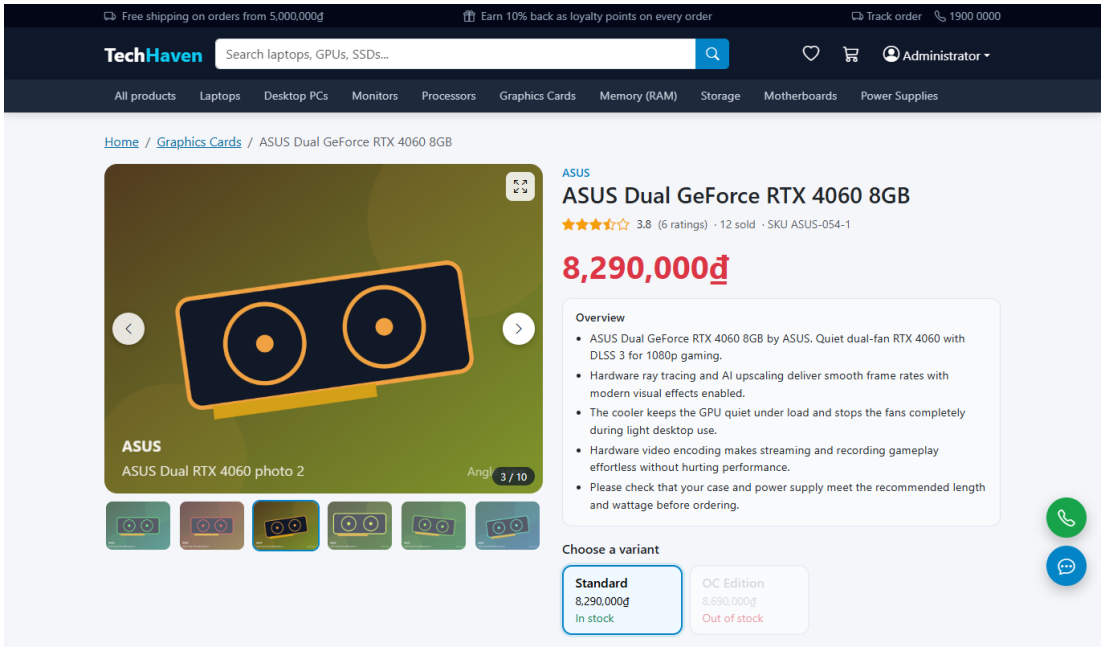


Figure A.8. *Product page with a 10-image gallery and variants*

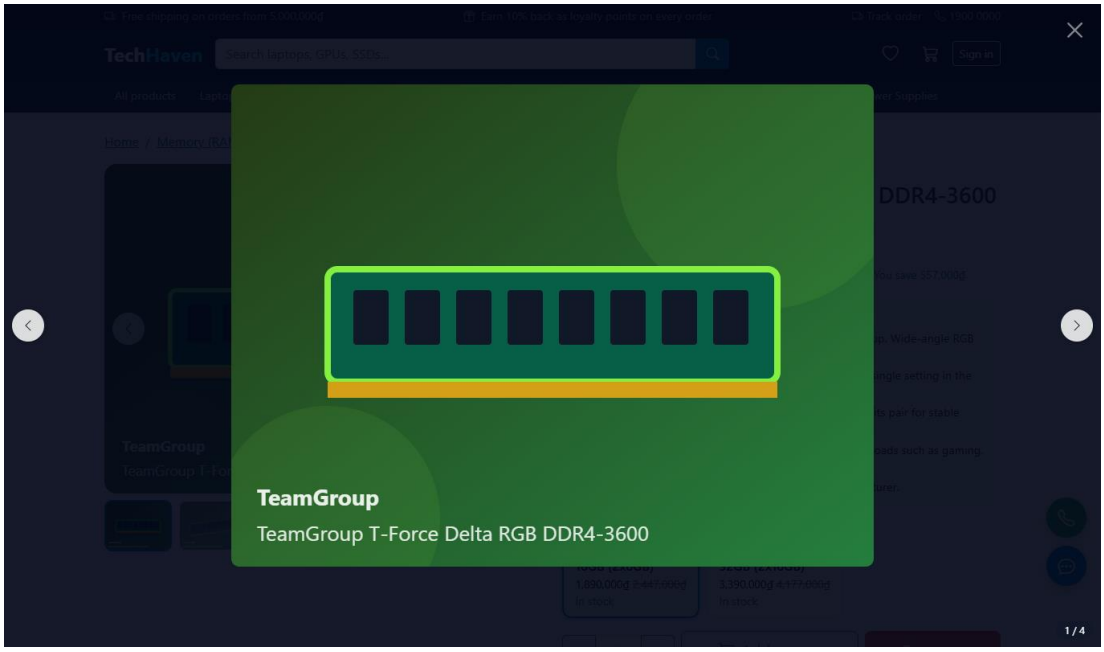


Figure A.9. *Full-screen image viewer*

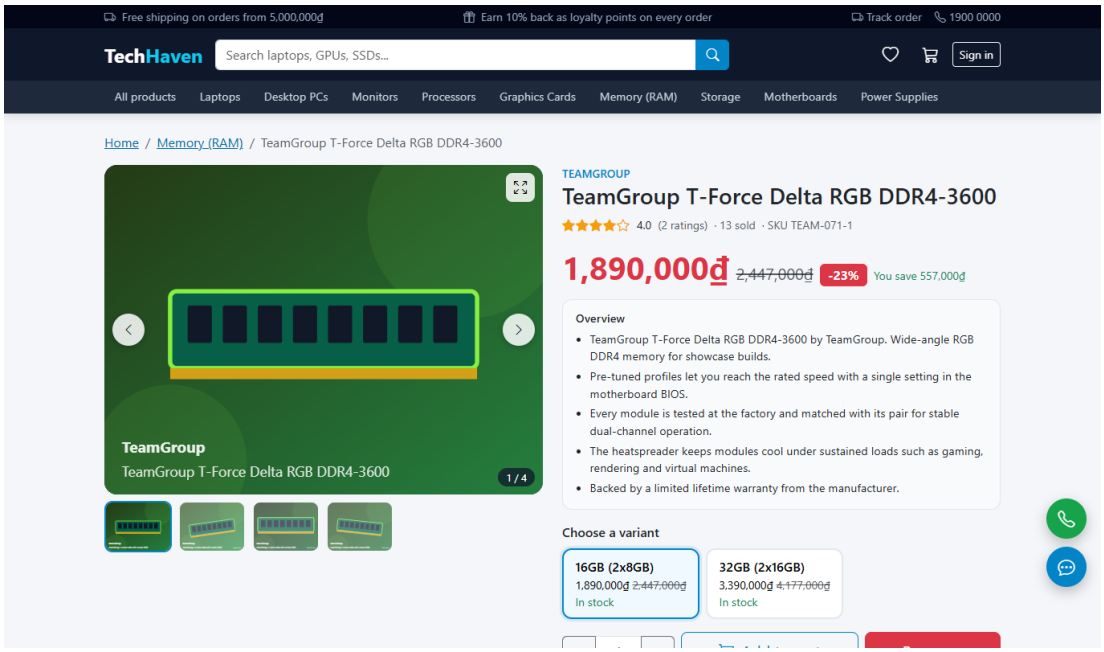


Figure A.10. *Original price, discount badge and five-line overview*

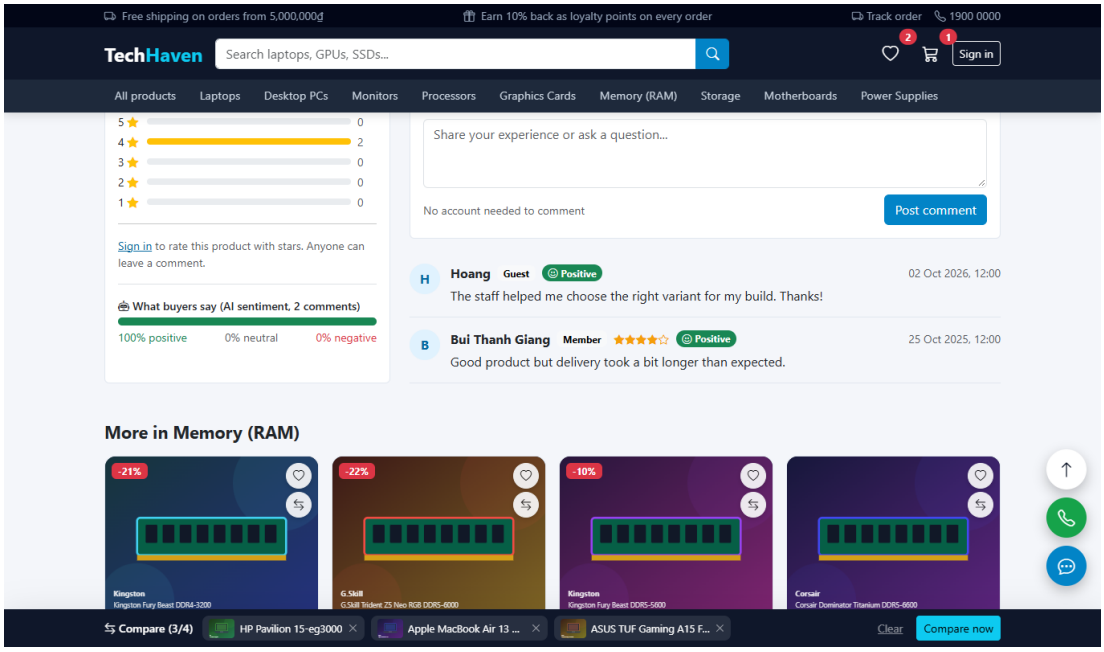


Figure A.11. *Ratings, AI sentiment summary and labelled comments*

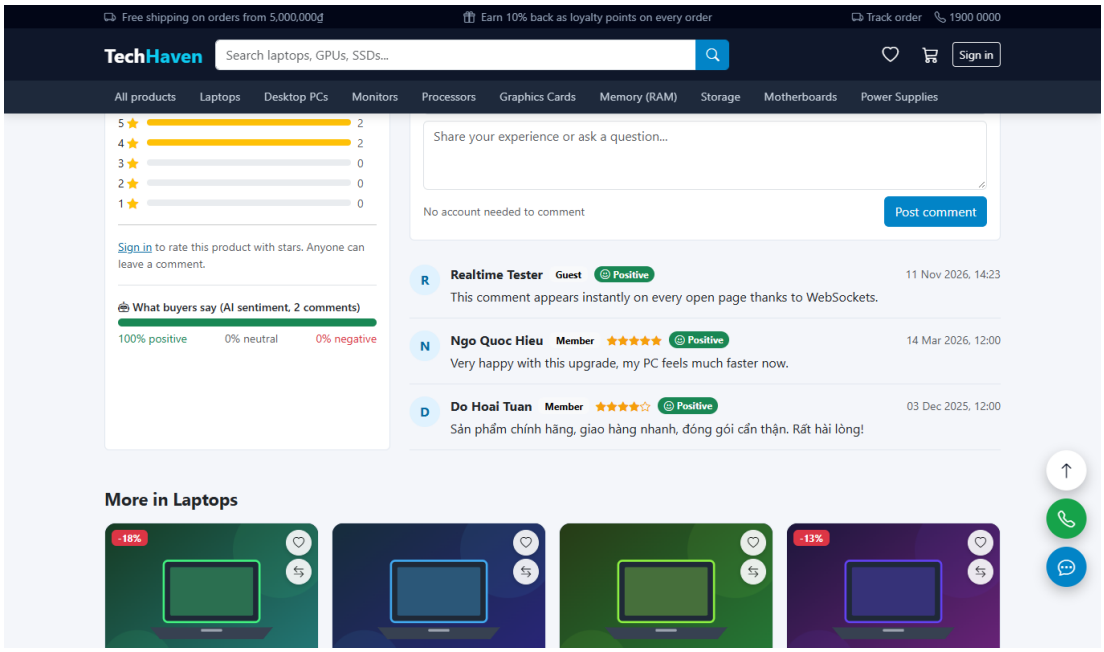


Figure A.12. *A comment from another browser appears instantly (WebSocket)*

A.4 Cart and Checkout

Adding a product opens the mini-cart (Figure A.13). The cart updates totals without reloading and shows the effect of a discount code before payment (Figure A.14). Checkout has three steps; a signed-in customer has saved addresses with the default pre-selected and can spend loyalty points (Figure A.15 to Figure A.19). The

success screen shows all order information and, for a guest, explains that an account was created (Figure A.20).

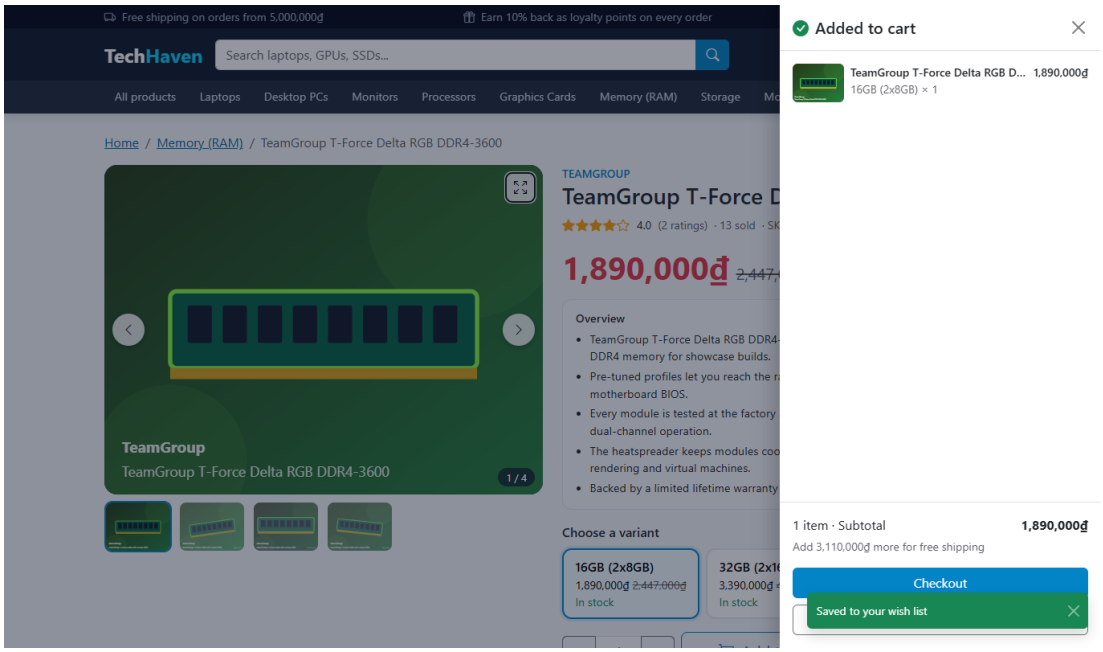


Figure A.13. *Mini-cart after "Add to cart"*

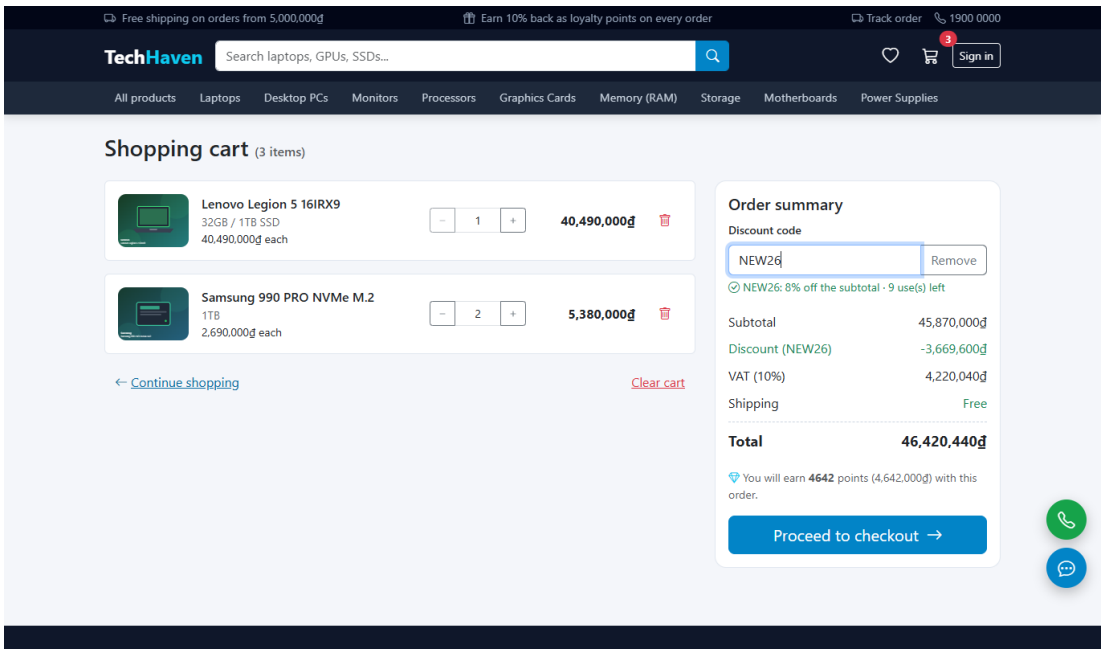


Figure A.14. *Cart with a valid discount code*

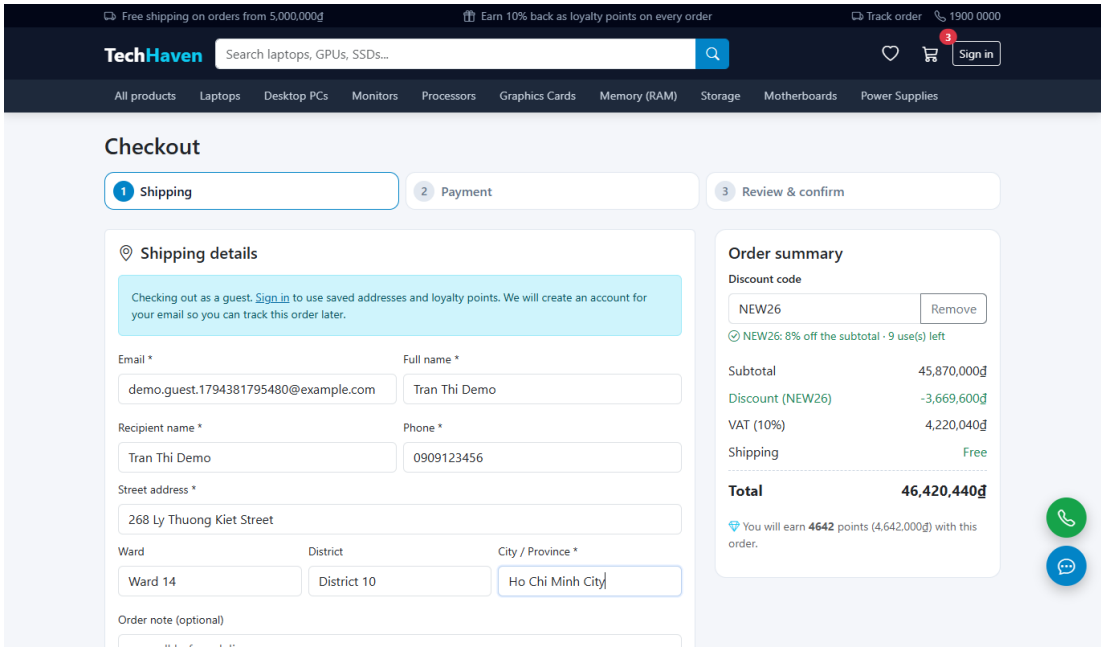


Figure A.15. *Checkout step 1 - shipping (guest)*

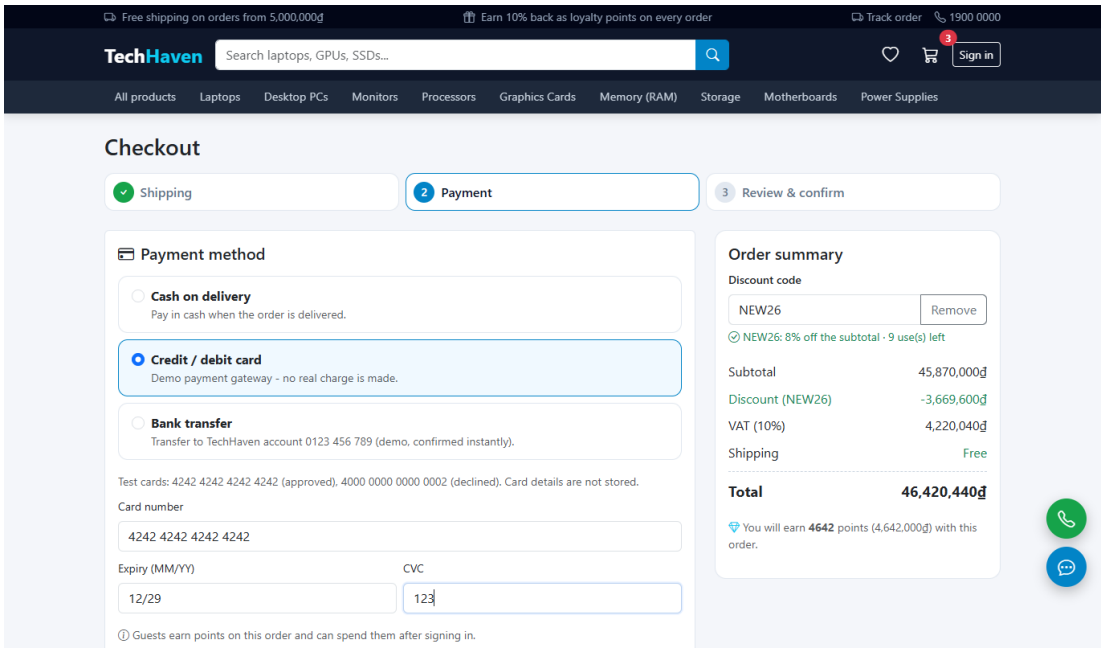


Figure A.16. *Checkout step 2 - payment*

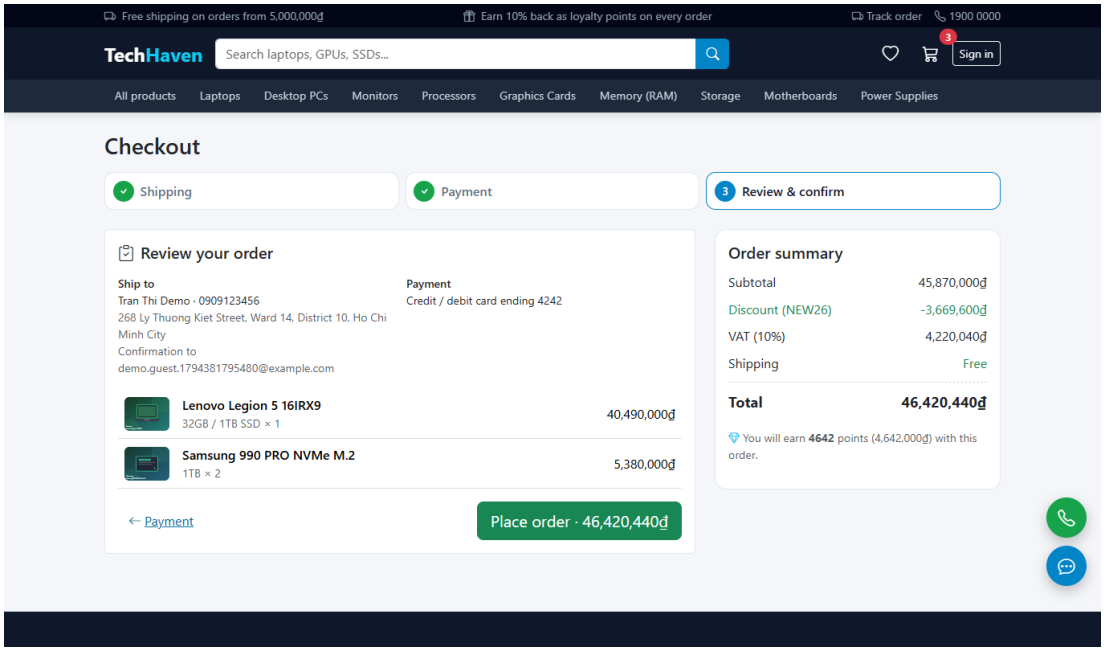


Figure A.17. *Checkout step 3 - review and confirm*

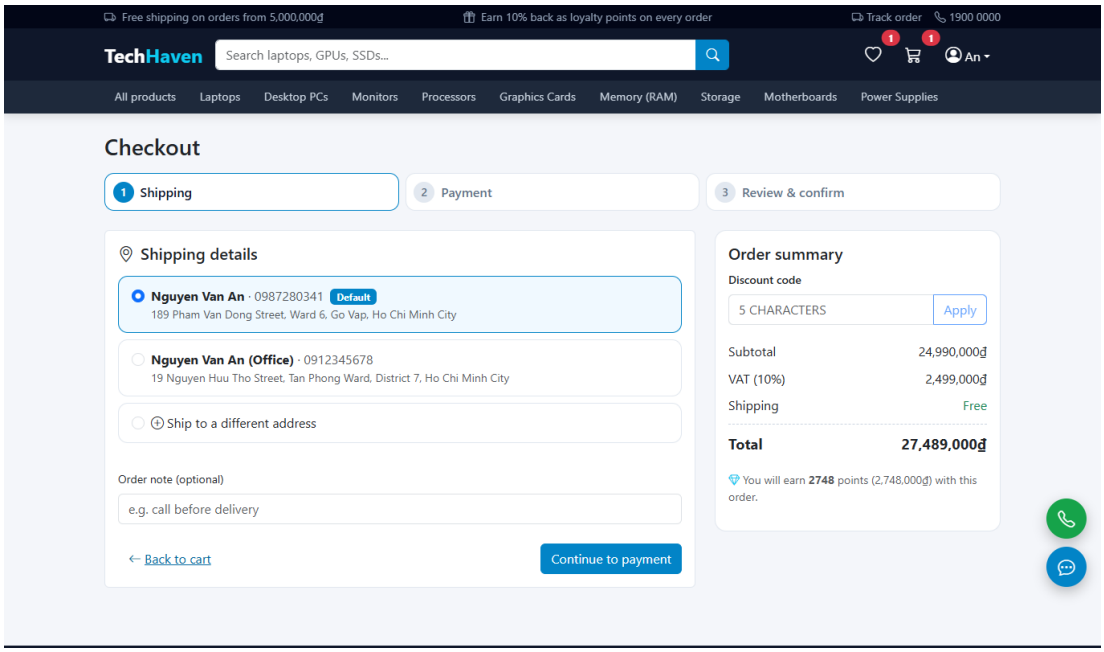


Figure A.18. *Saved addresses with the default pre-selected*

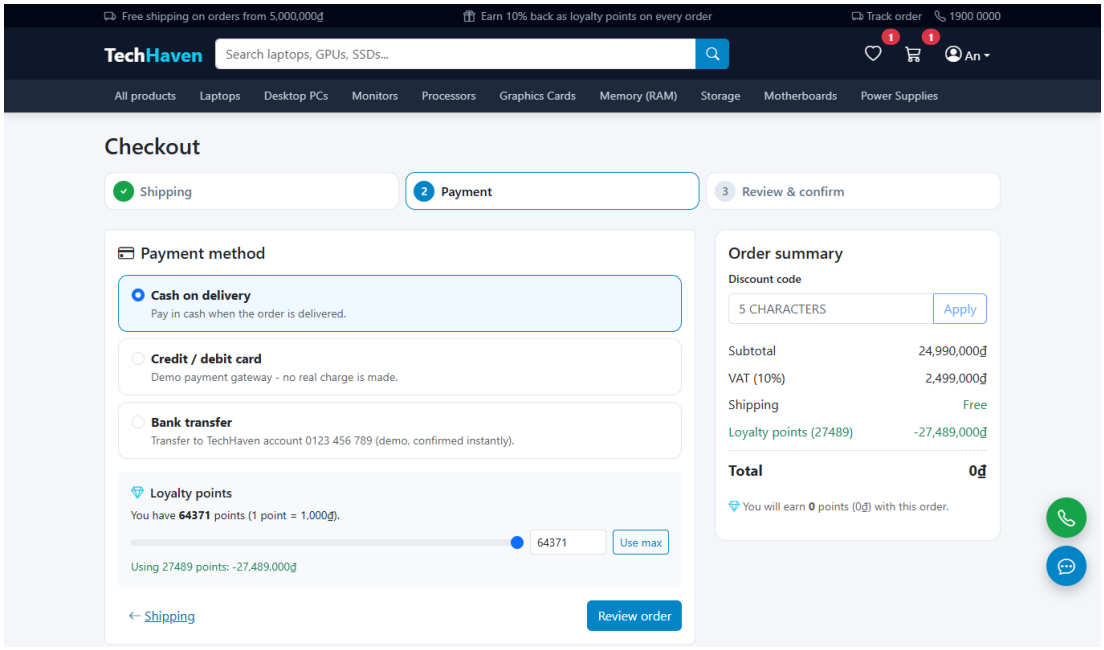


Figure A.19. *Spending loyalty points*

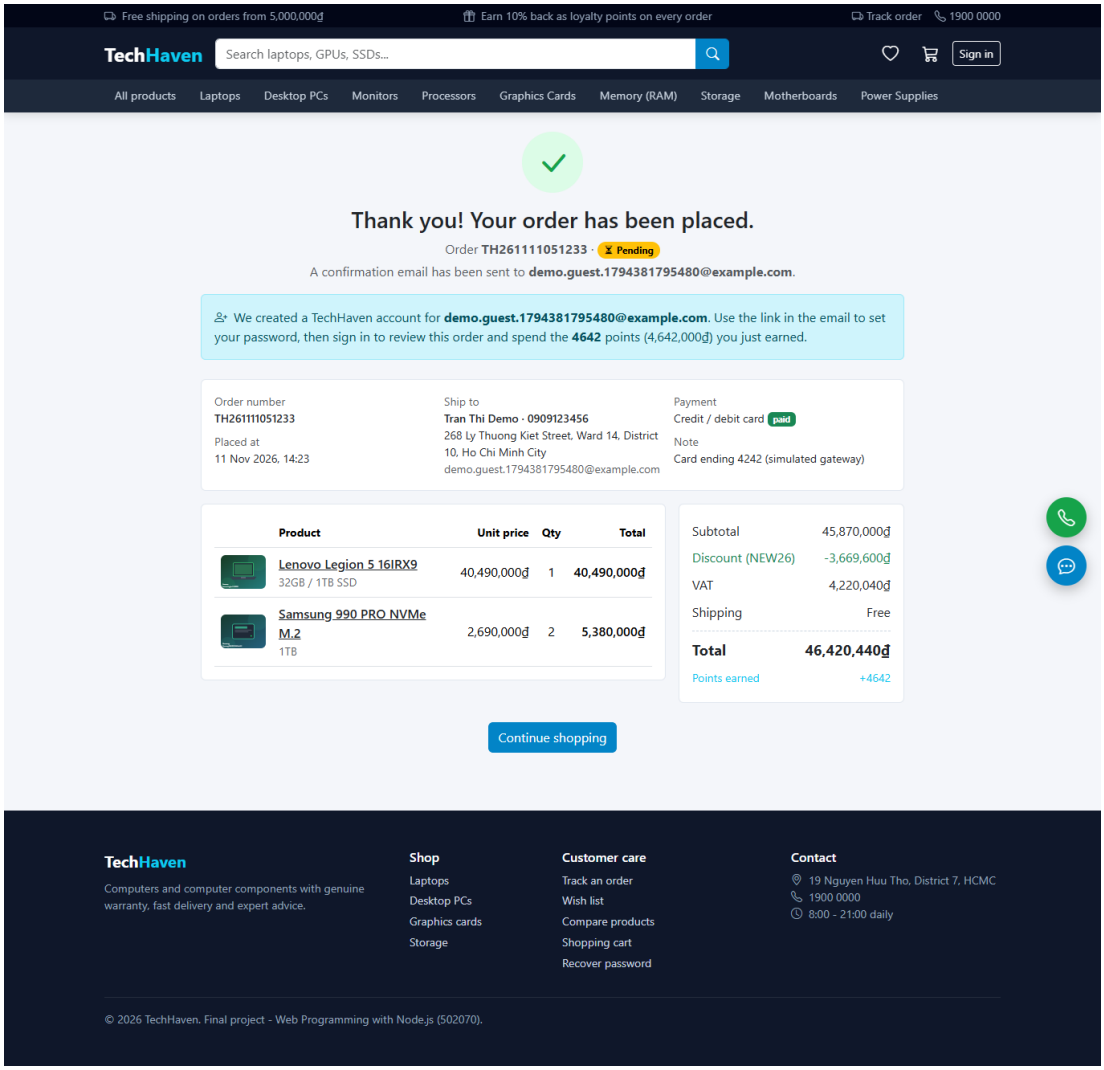


Figure A.20. *Order success screen after a guest checkout*

A.5 Accounts, Orders and Shopping Aids

Registration asks only for e-mail, full name and address; sign-in offers e-mail and password or Google; recovery sends a reset link (Figure A.21 to Figure A.23). In these screenshots Google sign-in is shown as not configured because no OAuth client was set. The profile page manages personal data, the password and several addresses; the order pages show the history and the status timeline, newest first (Figure A.24 to Figure A.26). Any order can also be tracked with its number and e-mail (Figure A.27), and products can be compared or saved to the wish list (Figure A.28 to Figure A.30).

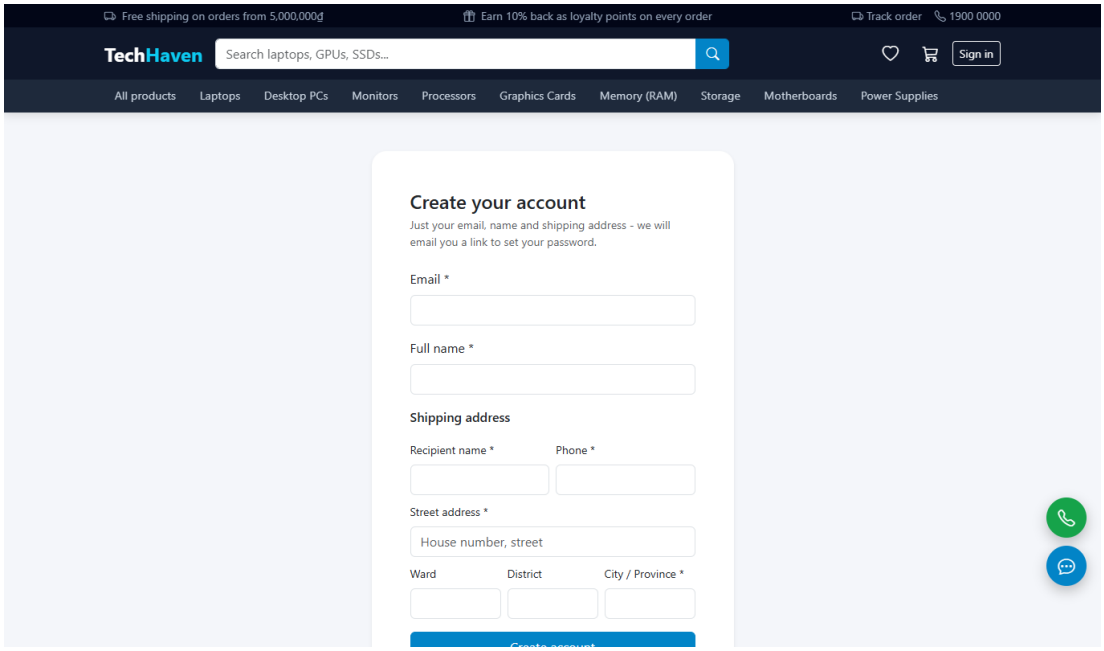


Figure A.21. *Registration with e-mail, full name and address*

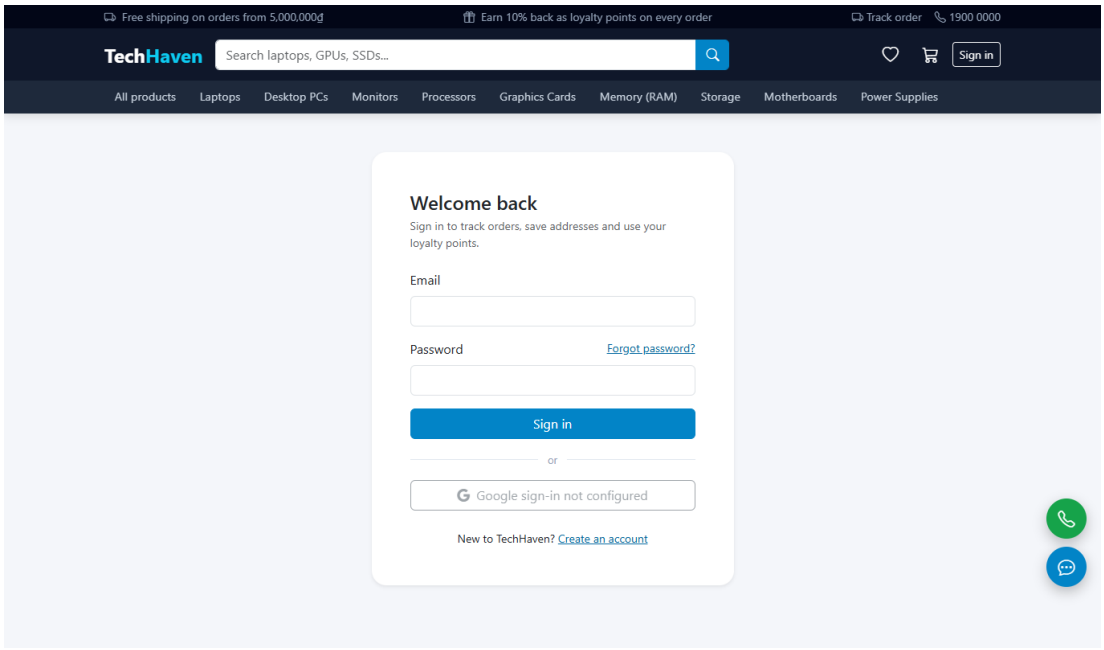


Figure A.22. *Sign-in page*

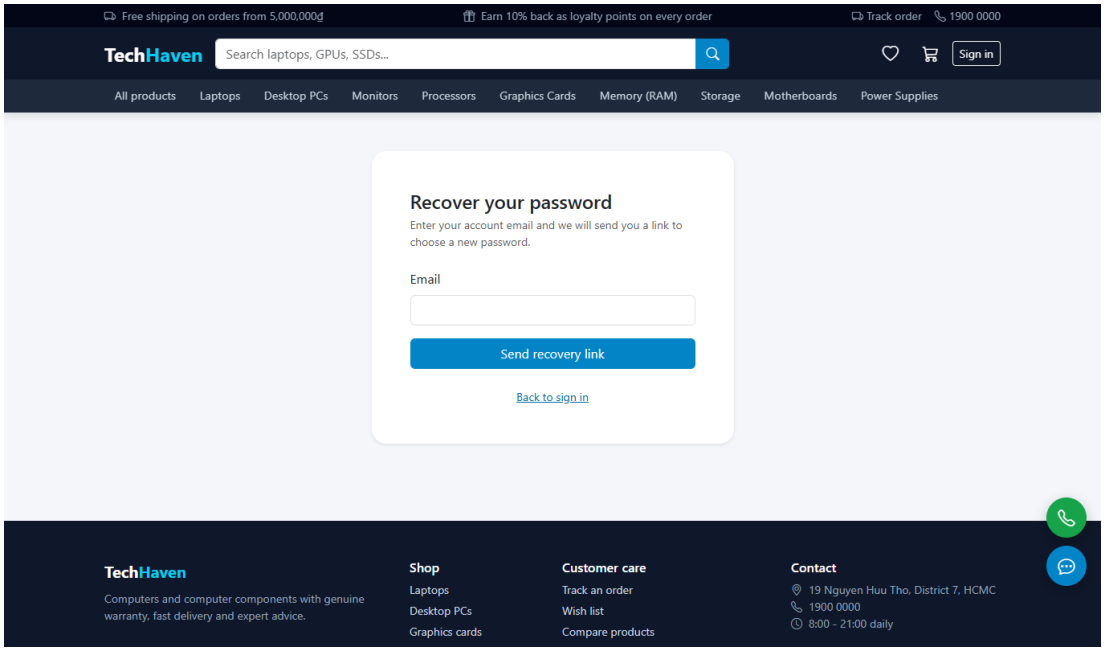


Figure A.23. *Password recovery*

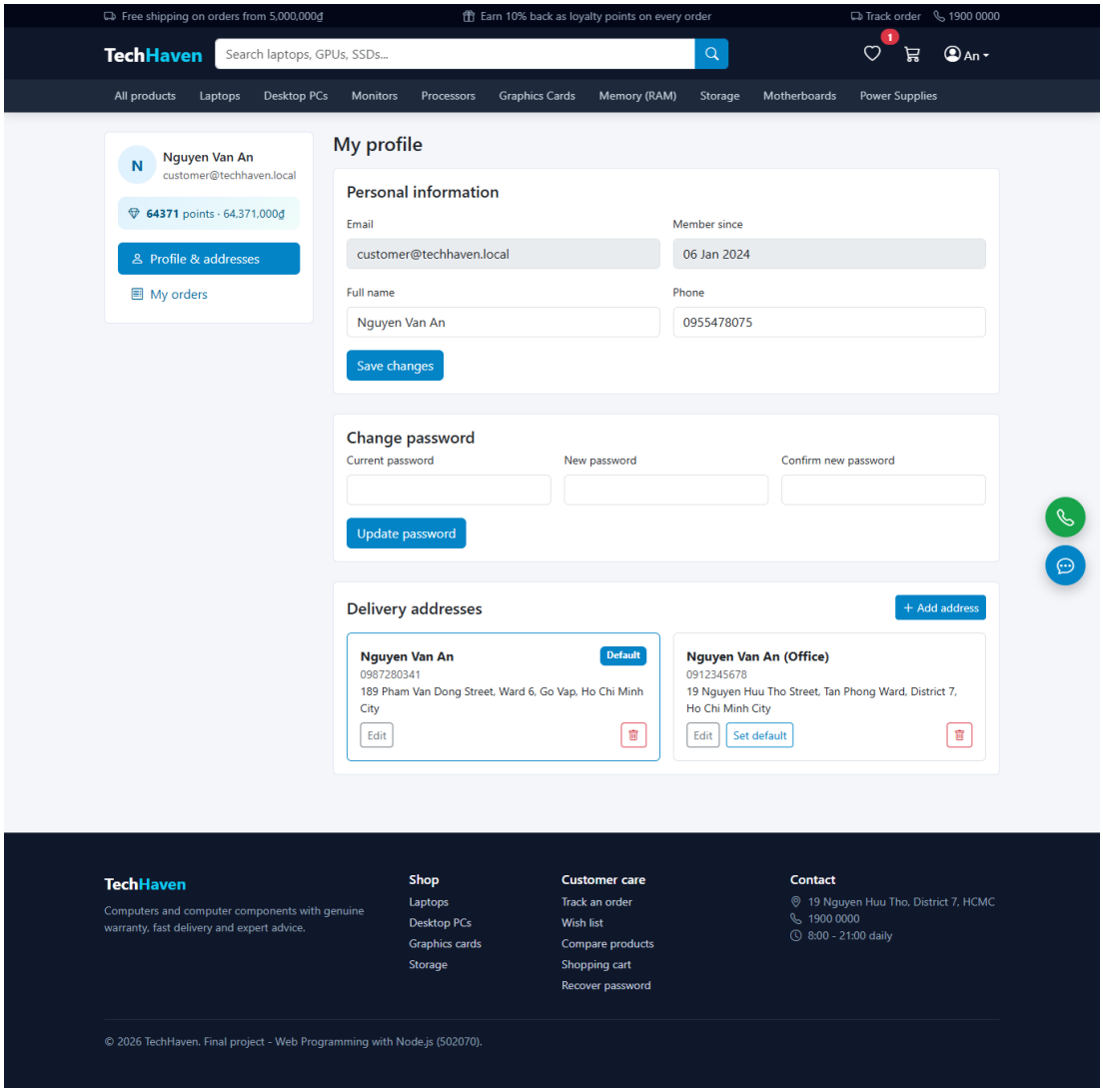


Figure A.24. *Profile, password and addresses*

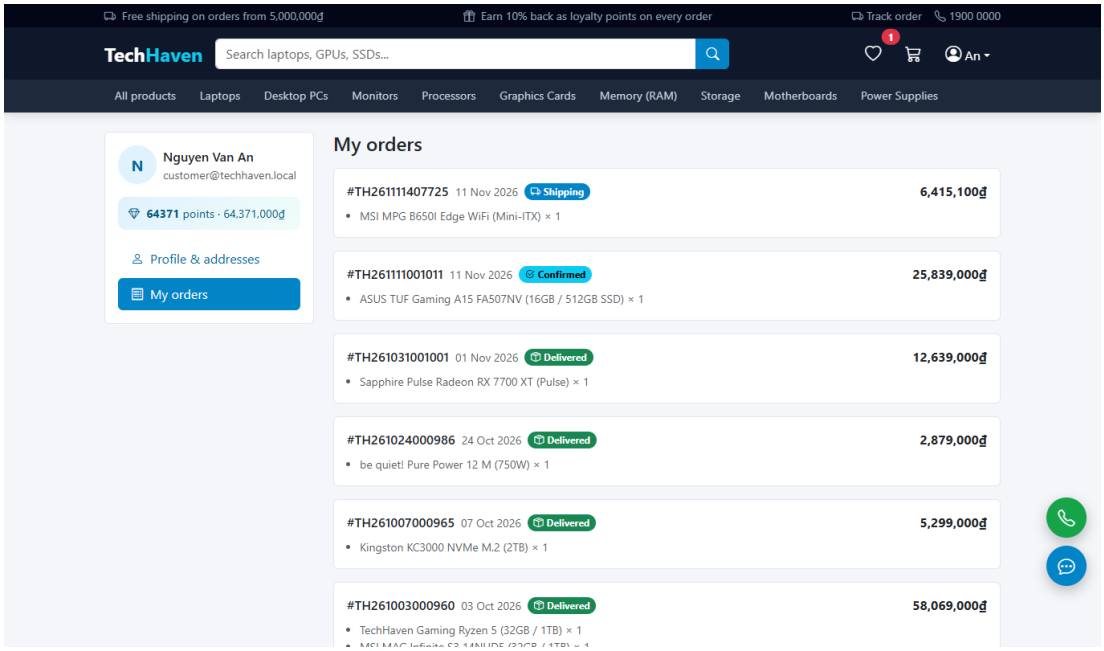


Figure A.25. *Order history*

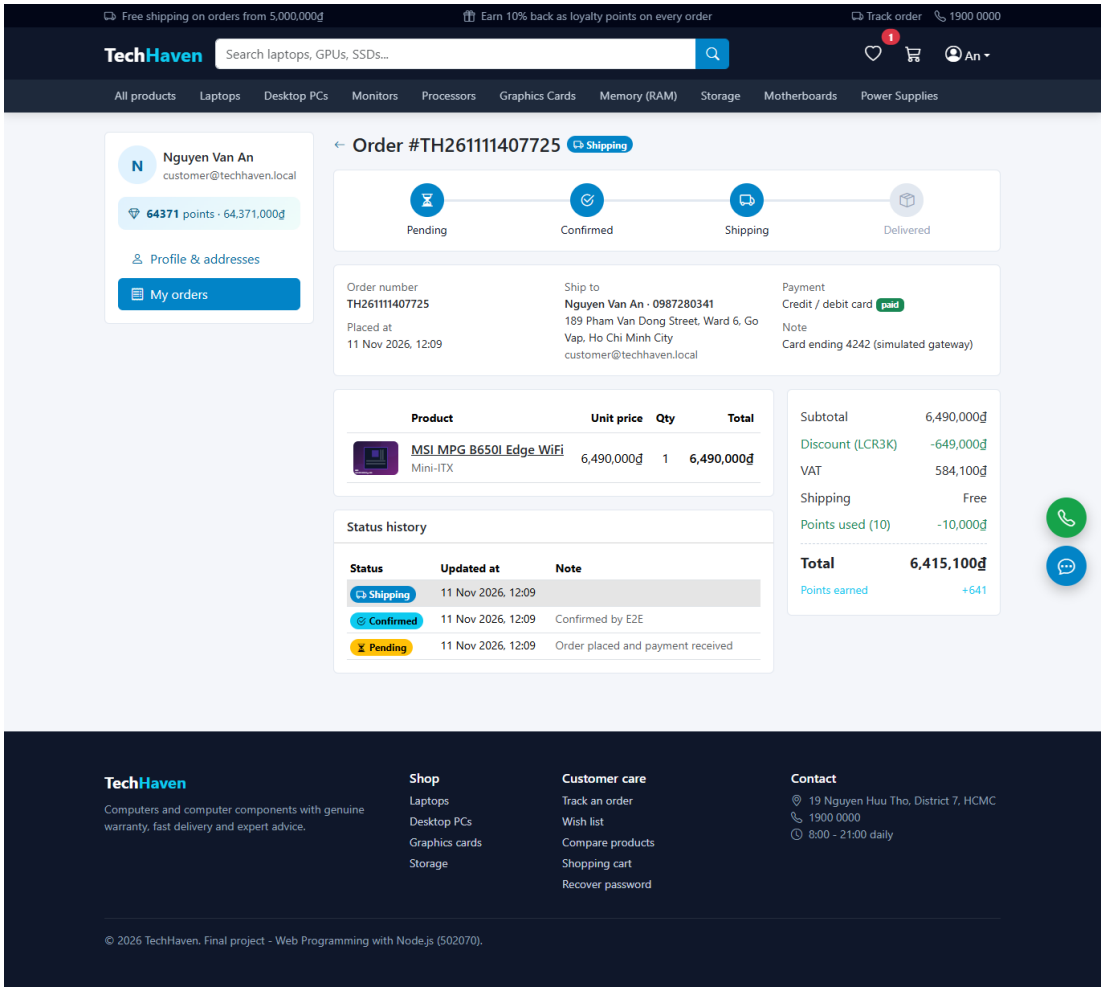


Figure A.26. *Order details with status history*

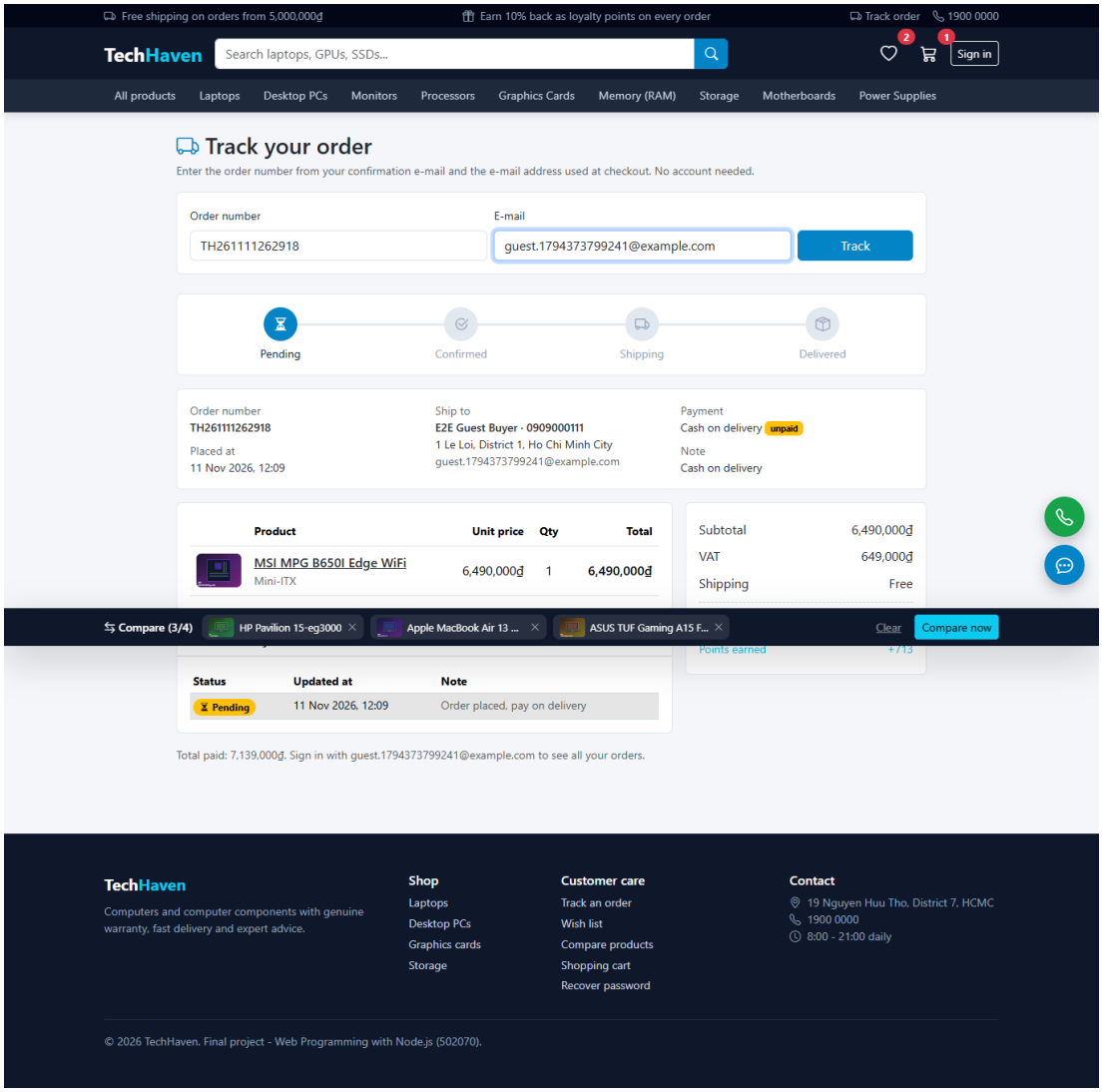


Figure A.27. *Tracking an order without signing in*

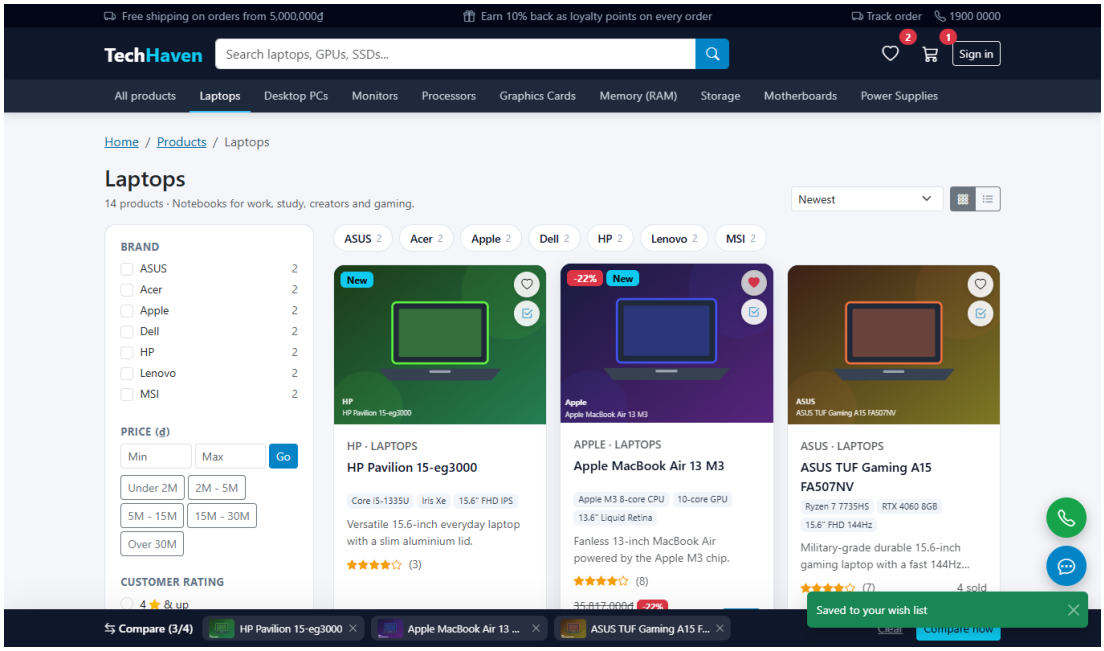


Figure A.28. *Products selected for comparison*

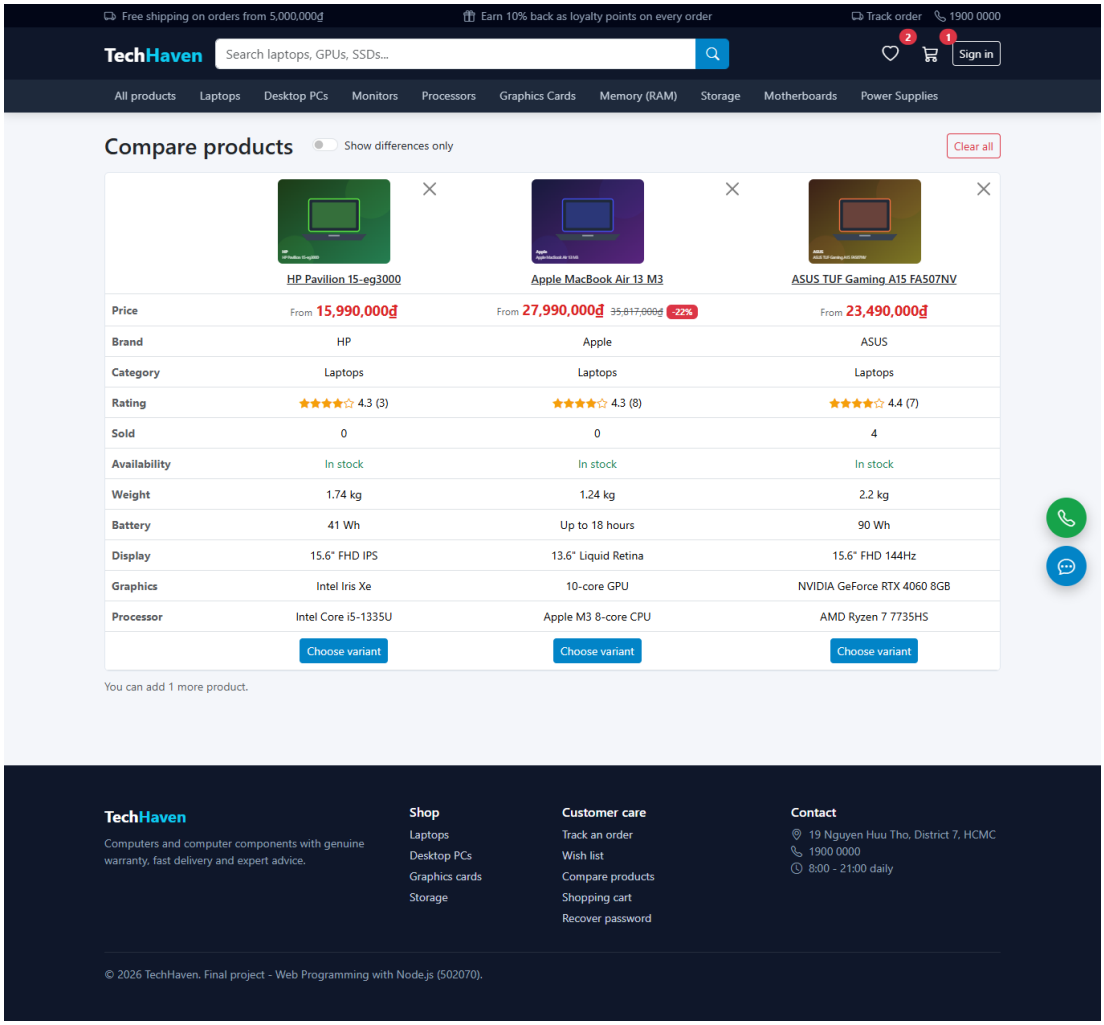


Figure A.29. *Comparison of three laptops*

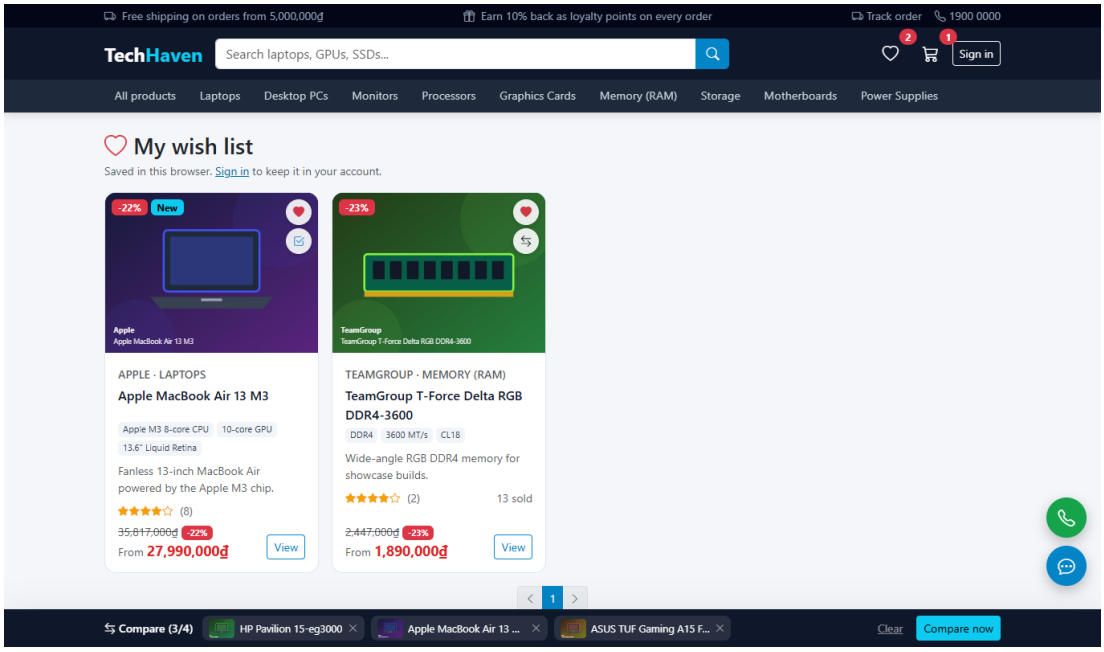


Figure A.30. *Wish list*

A.6 Administrator Console

The product form contains the image manager: Figure A.31 shows it with four uploaded images and Figure A.32 after reaching the limit of ten, with the cover moved to another image, the upload area disabled and the eleventh file skipped. Each variant has an optional original price (Figure A.33). The other figures show categories, the simple dashboard with its "Needs attention" insights, the advanced dashboard, orders, users and discount codes (Figure A.34 to Figure A.43).

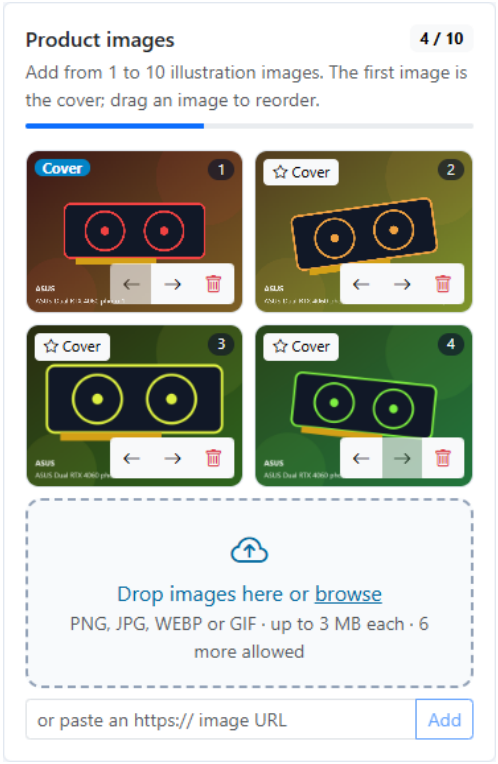


Figure A.31. *Image manager with 4 of 10 images*

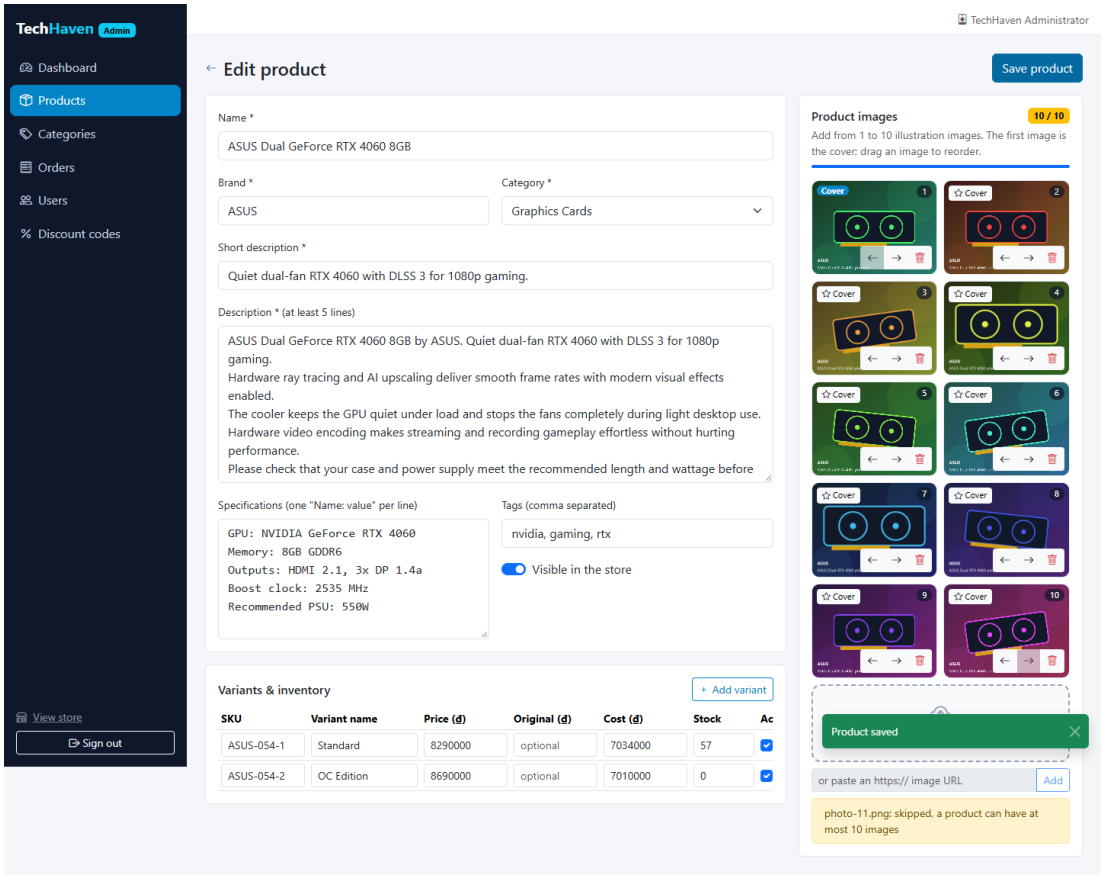


Figure A.32. *Product form with 10 of 10 images; the 11th file is skipped*

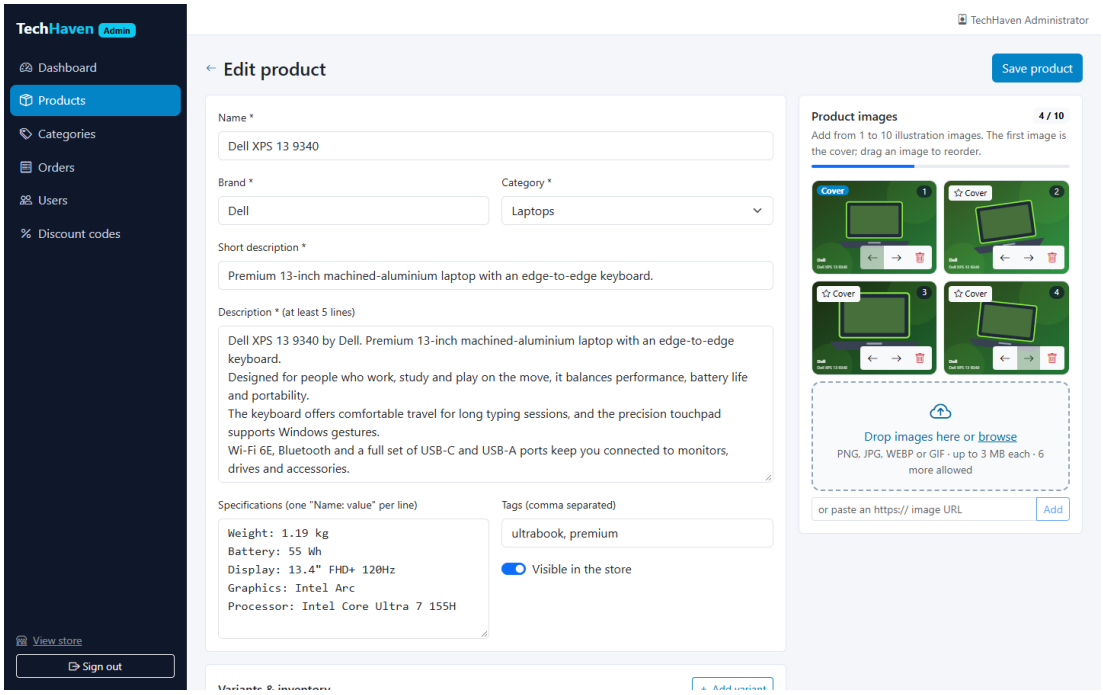


Figure A.33. *Variants with selling price, original price, cost and stock*

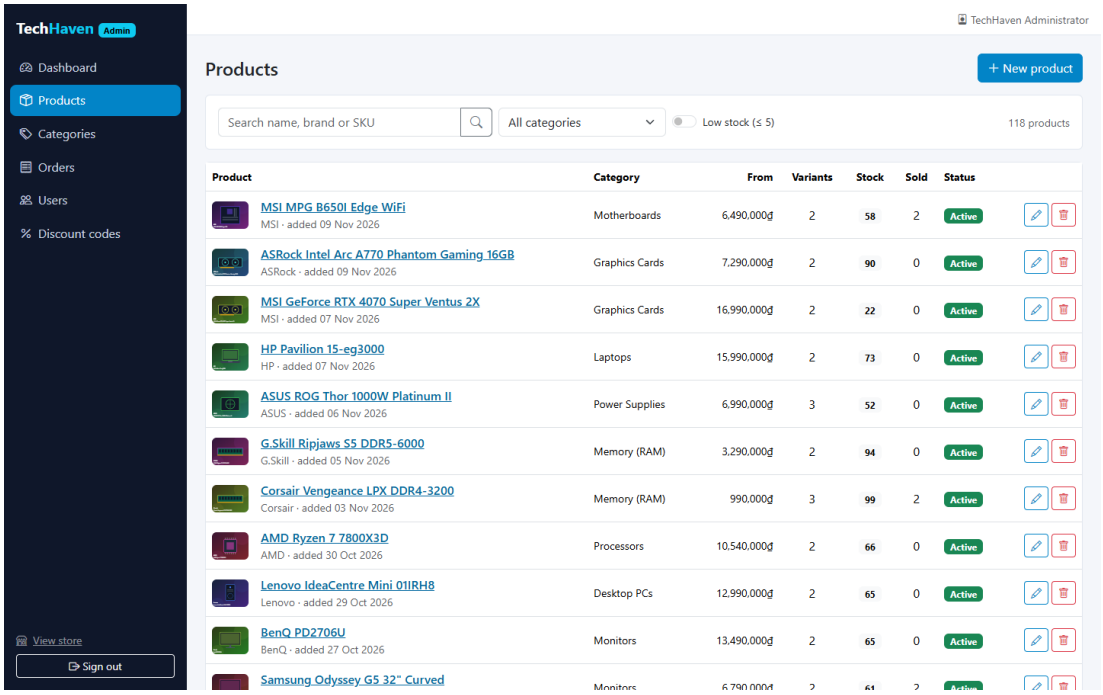


Figure A.34. *Product management*

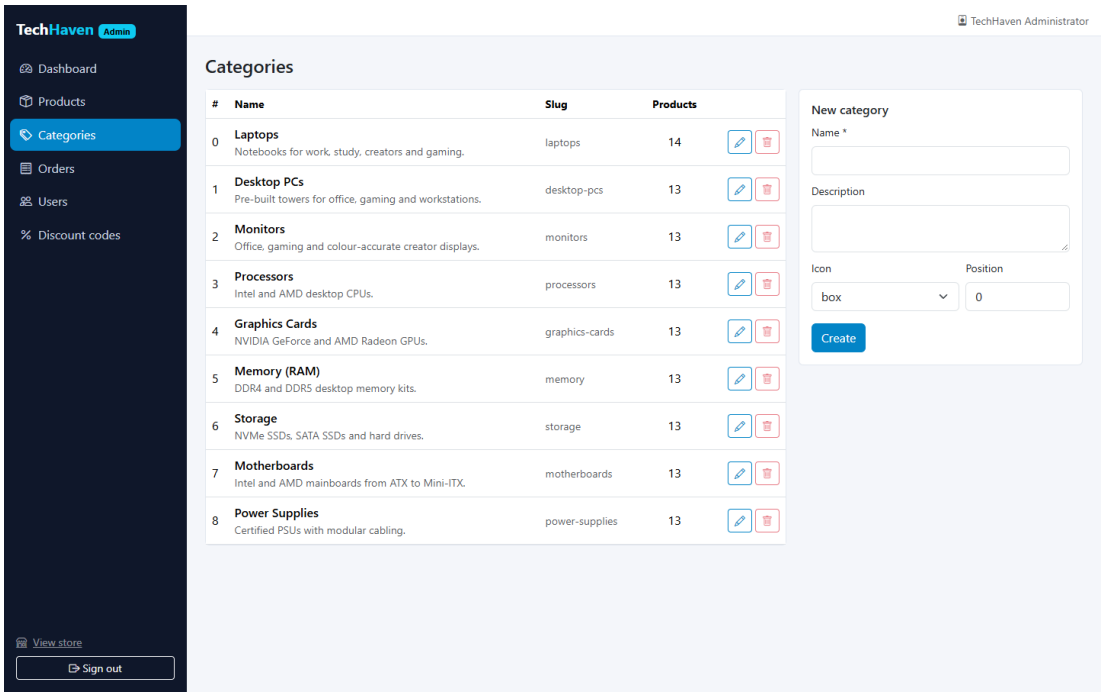


Figure A.35. *Category management*

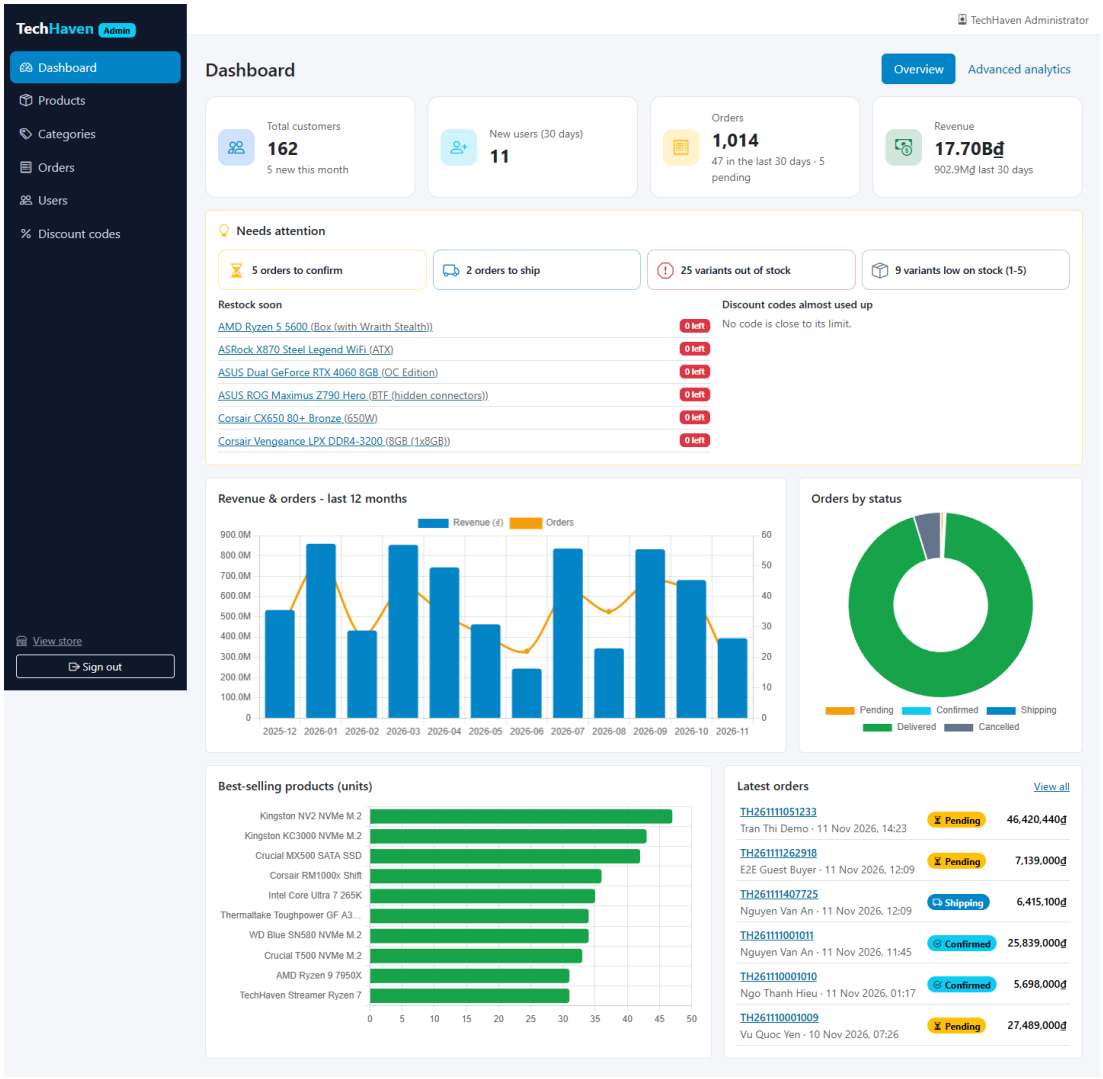


Figure A.36. *Simple dashboard*

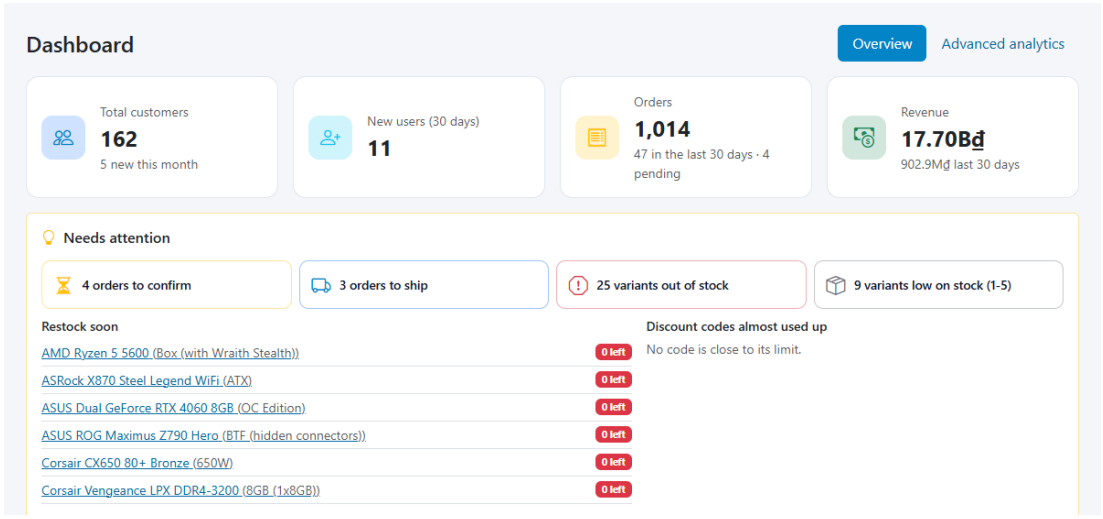


Figure A.37. *Actionable insights on the simple dashboard*

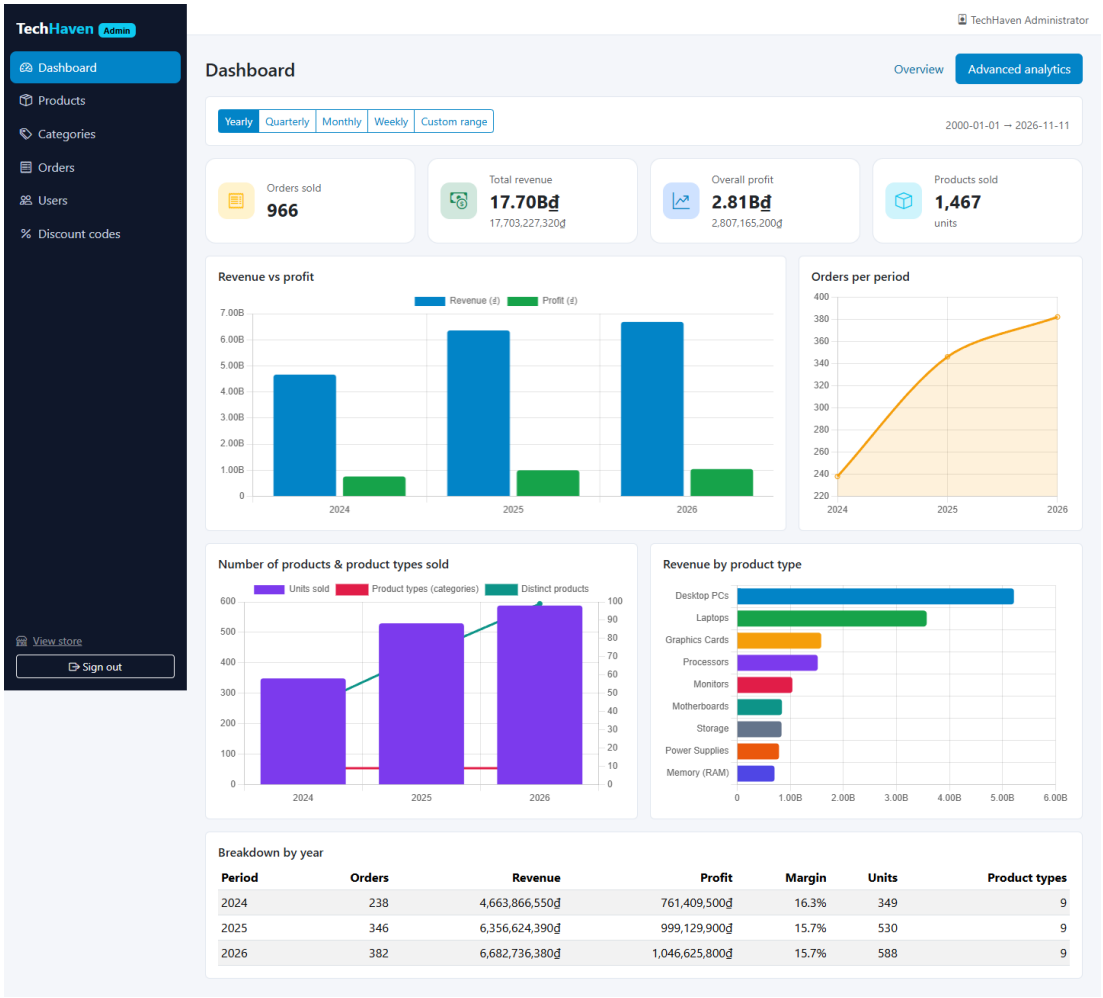


Figure A.38. *Advanced dashboard - yearly view (default)*

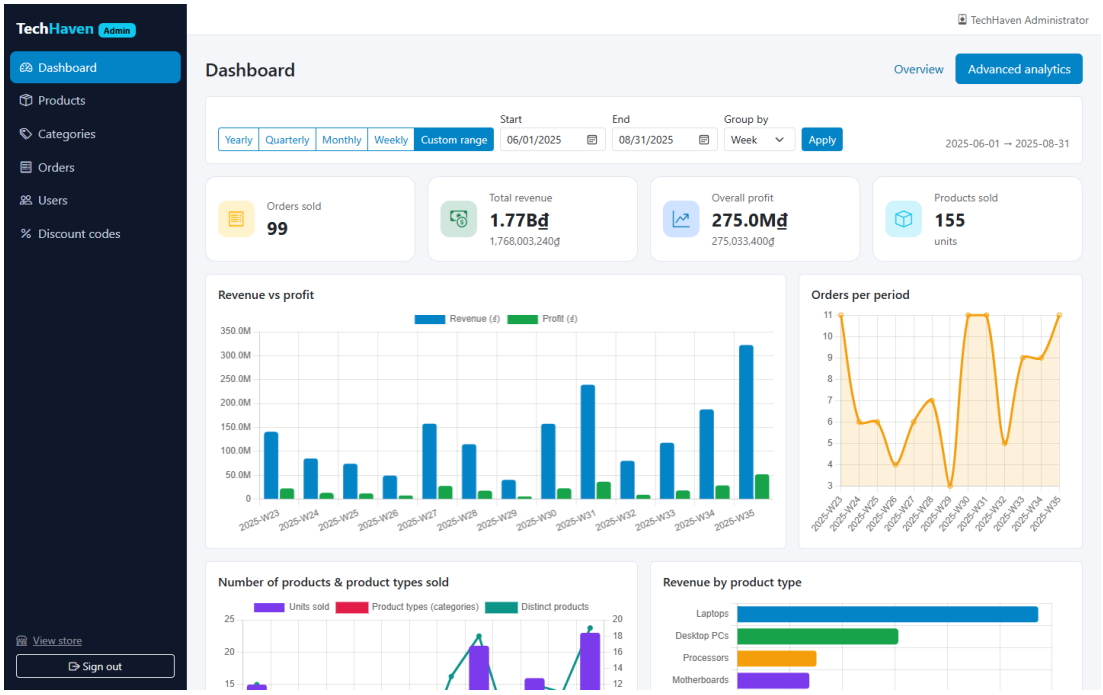


Figure A.39. *Advanced dashboard - custom range grouped by week*

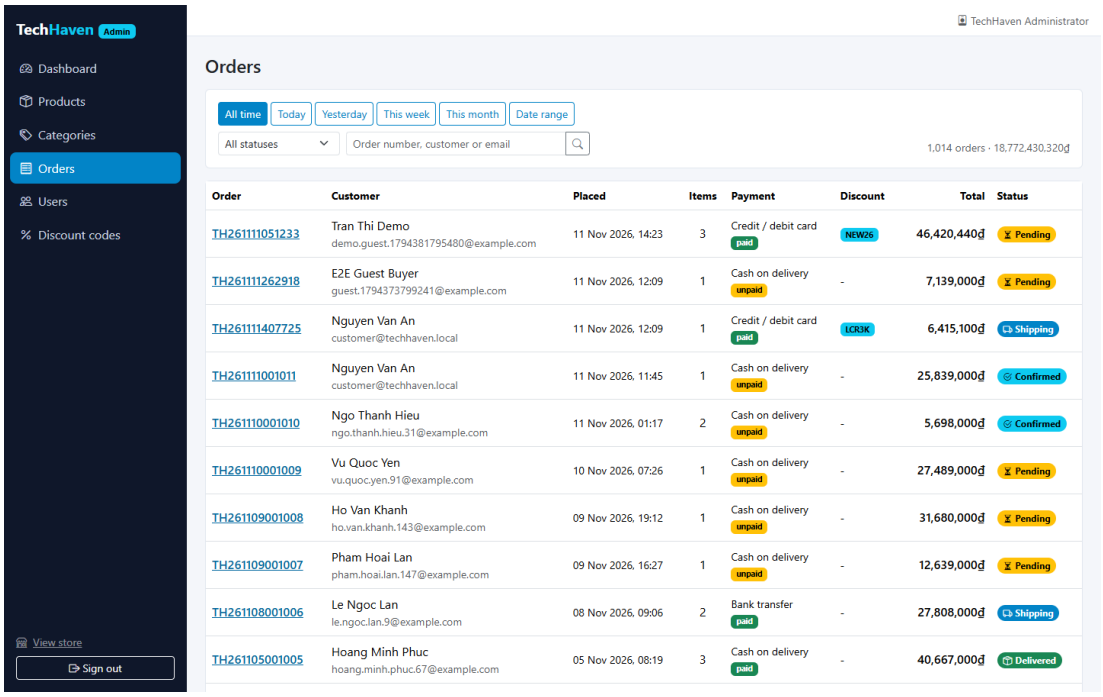


Figure A.40. *Order list, newest first, 20 per page*

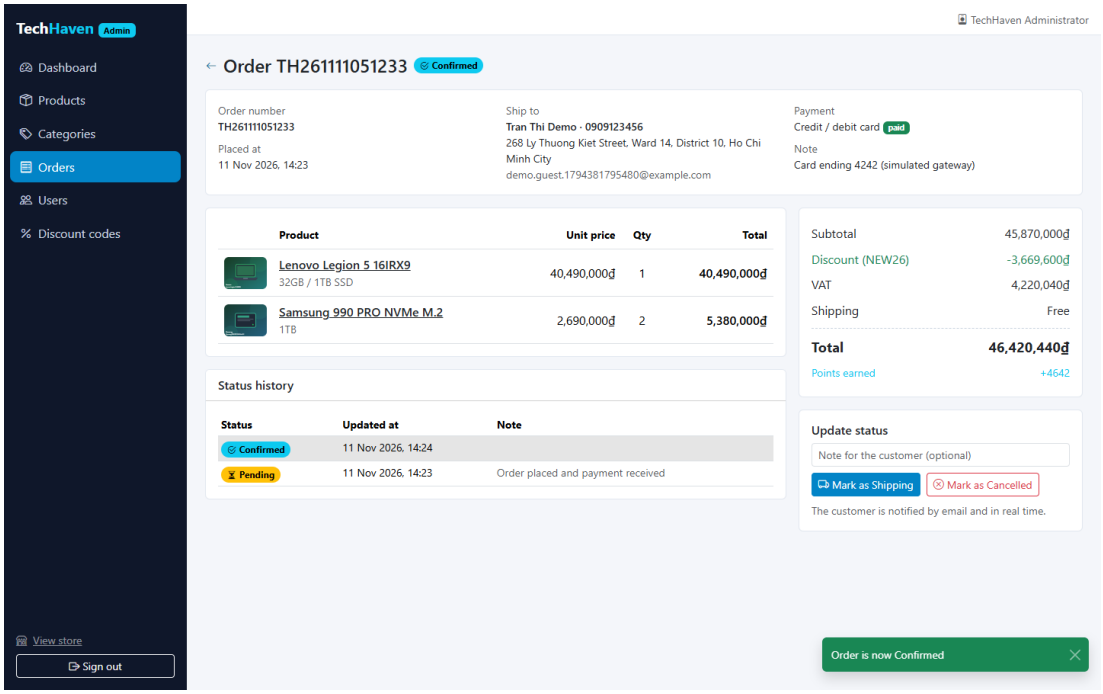


Figure A.41. *Order details and status update*

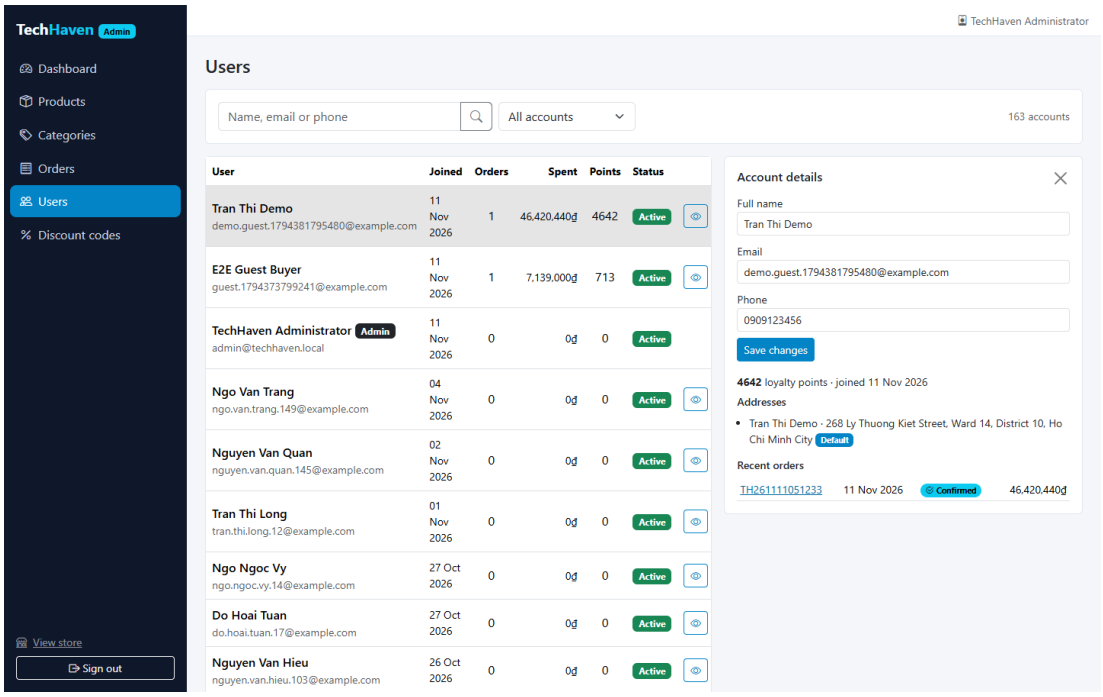


Figure A.42. *User management*

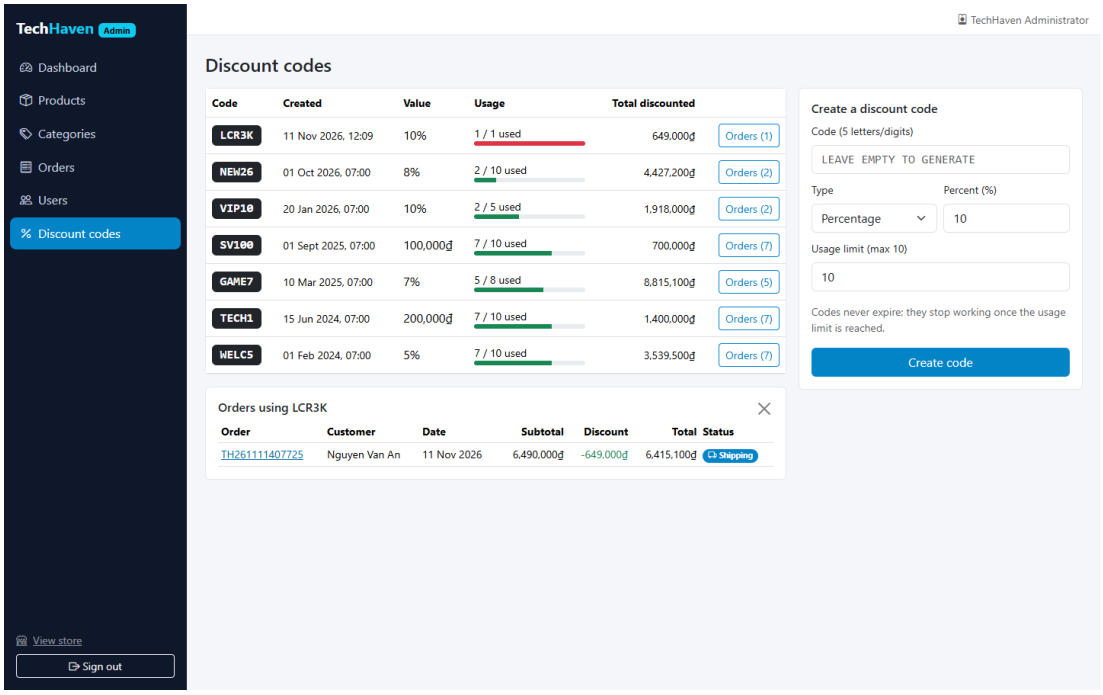


Figure A.43. *Discount codes with usage and orders*

A.7 AI Assistant, E-mail, Responsive Design and Infrastructure

The AI assistant answers in Vietnamese and English with products from the catalogue (Figure A.44, Figure A.45). Every e-mail sent by the shop appears in the Mailpit inbox (Figure A.46, Figure A.47). Figure A.48 to Figure A.51 show the store on a phone (390 × 844) and a tablet (820 × 1180), and Figure A.52 the durable queues in RabbitMQ.

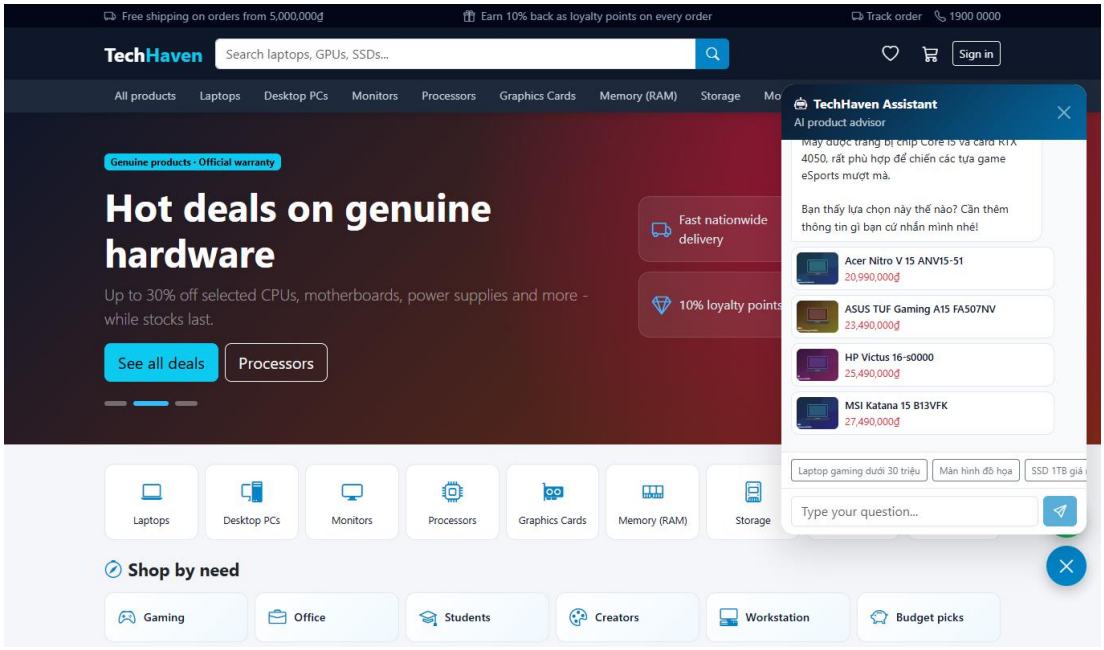


Figure A.44. *AI assistant (Gemini): gaming laptops under 30 million VND and a follow-up question (Vietnamese)*

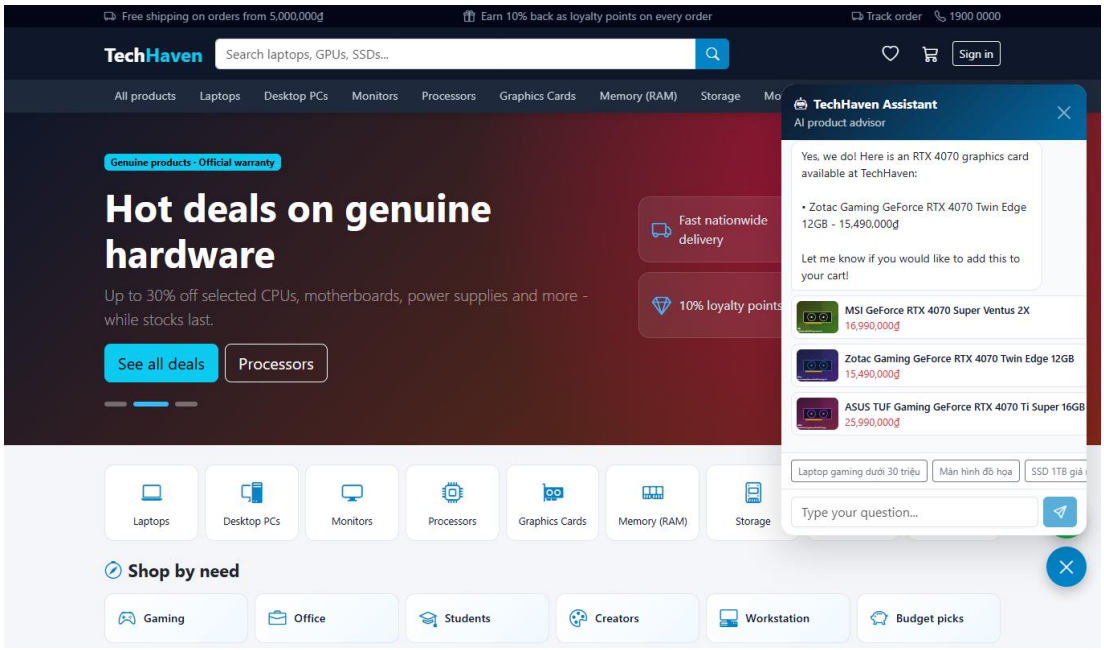


Figure A.45. *AI assistant (Gemini): a specific graphics card (English)*

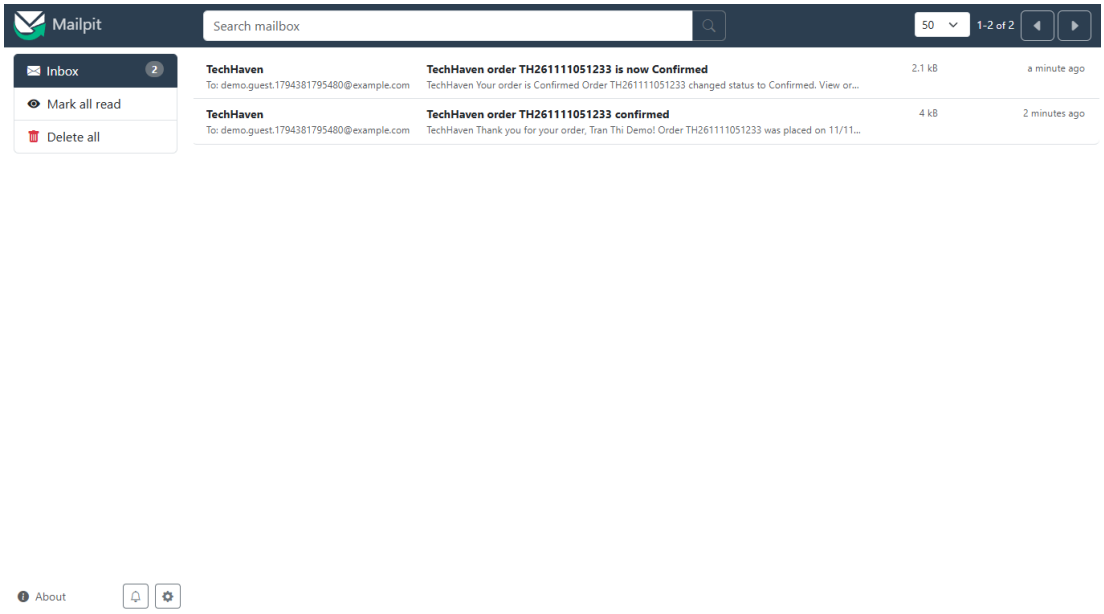


Figure A.46. *Mailpit inbox with the e-mails sent by the shop*

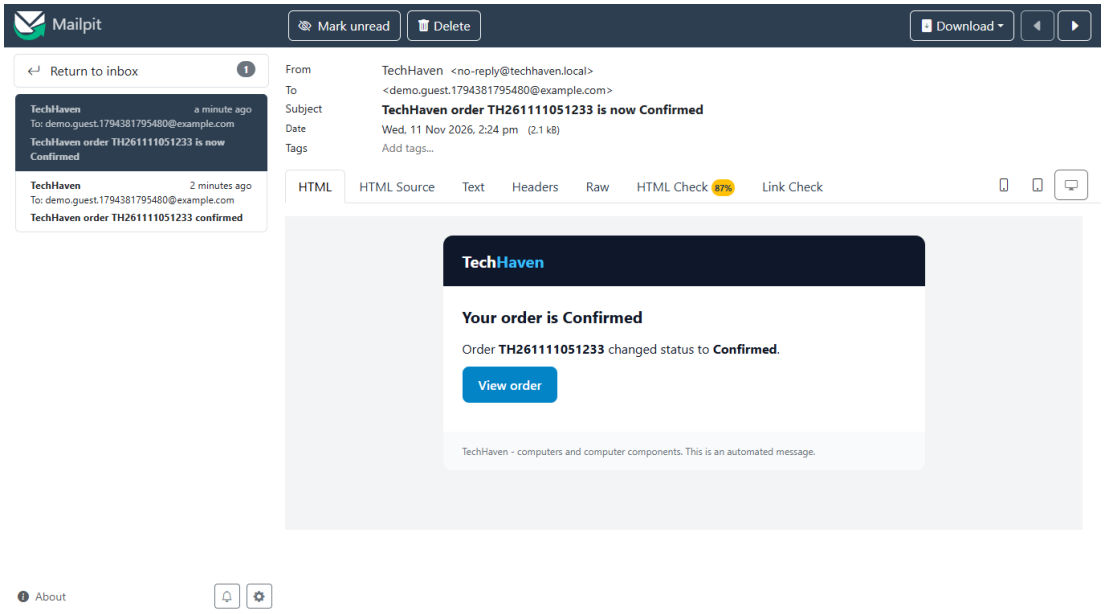


Figure A.47. *Order confirmation e-mail*

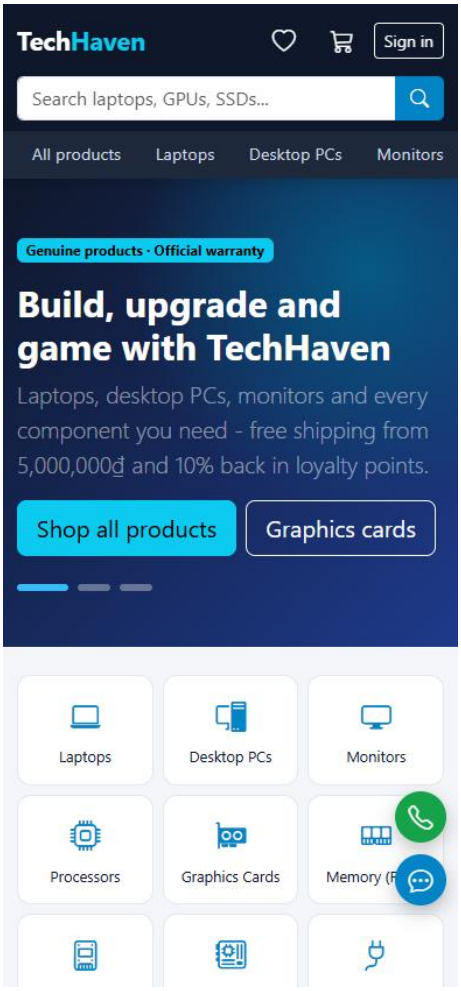


Figure A.48. *Landing page on a phone*



Figure A.49. *Product page on a phone*

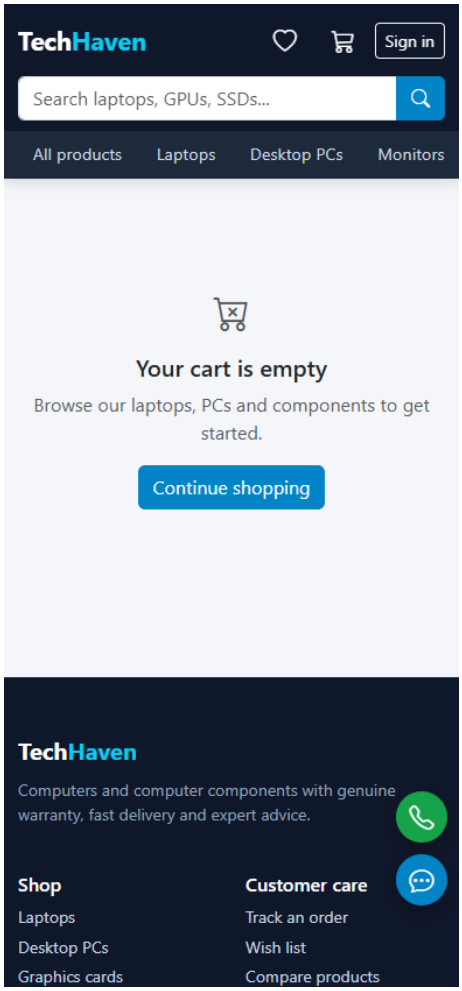


Figure A.50. *Cart on a phone*

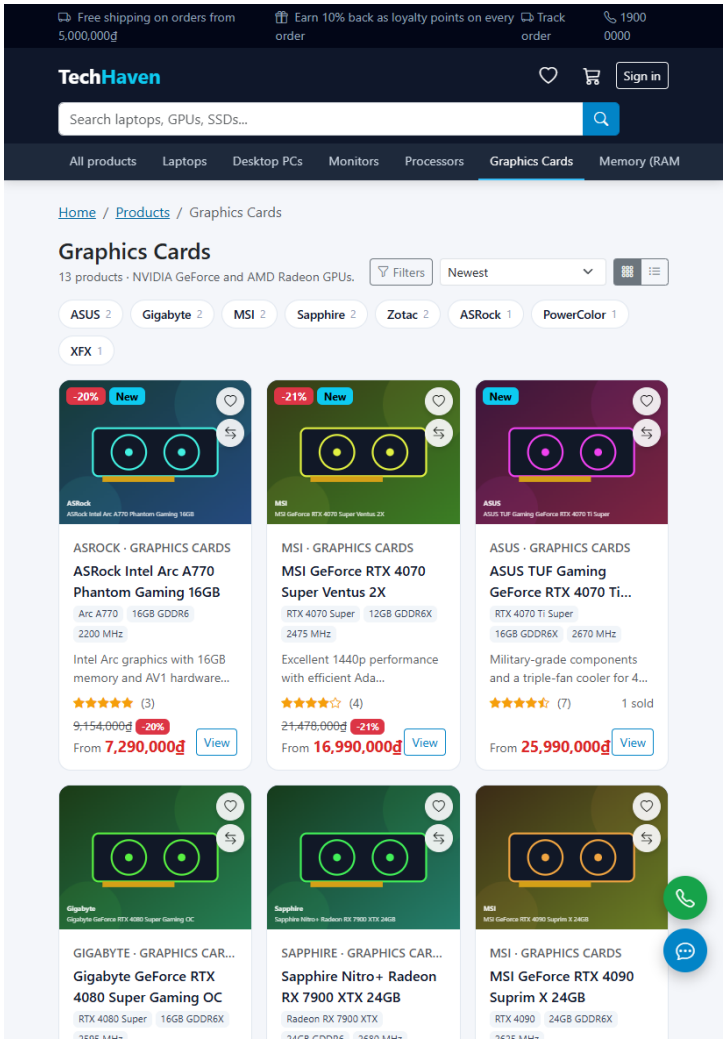


Figure A.51. *Catalogue on a tablet*

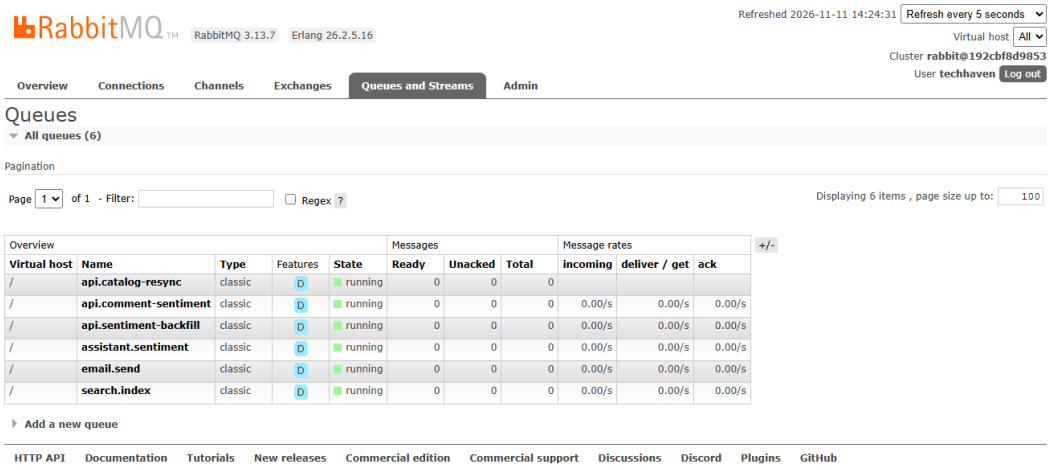


Figure A.52. *Durable queues in RabbitMQ*

APPENDIX B. COMPLETE TEST RESULTS

Generated from Bonus/unit-test-output.txt and Bonus/e2e-test-output.txt (final run: 25 of 25 unit tests and 66 of 66 end-to-end checks passed).

Table B.1. *Complete test results*

ID	Type	Test case	Result
U01	Unit	image type is detected from the file signature	Pass
U02	Unit	a product needs between 1 and 10 images	Pass
U03	Unit	duplicates are removed and only safe sources are accepted	Pass
U04	Unit	loyalty: 1,000,000 VND earns 100 points worth 100,000 VND	Pass
U05	Unit	discount codes are exactly five alphanumeric characters	Pass
U06	Unit	percent and fixed discounts never exceed the subtotal	Pass
U07	Unit	quote: tax, shipping fee and total	Pass
U08	Unit	quote: free shipping above the threshold	Pass
U09	Unit	quote: points are capped by balance and by the order total	Pass
U10	Unit	quote: discount is applied before tax	Pass
U11	Unit	email is normalised and validated	Pass
U12	Unit	phone accepts 9-15 digits with separators	Pass
U13	Unit	password policy requires letters and digits	Pass
U14	Unit	slugify handles Vietnamese characters	Pass
U15	Unit	int enforces bounds and fallback	Pass
U16	Unit	order confirmation lists items, totals and escapes HTML	Pass
U17	Unit	guest checkout mail contains the set-password link	Pass
U18	Unit	every template renders	Pass
U19	Unit	sentiment: English comments	Pass
U20	Unit	sentiment: Vietnamese comments with negation	Pass
U21	Unit	budget parsing in both languages	Pass
U22	Unit	intent: category, usage, brand, model, FAQ and language	Pass
U23	Unit	intent: product words are not read as uses or policy questions	Pass
U24	Unit	LLM prompt: grounded turns and Gemini conversion	Pass
U25	Unit	follow-up questions keep the previous request	Pass
E01	End-to-end	requests are served by several API replicas	Pass
E02	End-to-end	landing page has New, Best Sellers and >= 3 category sections	Pass
E03	End-to-end	catalogue is paginated	Pass
E04	End-to-end	sort by name A-Z	Pass
E05	End-to-end	sort by price high-low	Pass
E06	End-to-end	filter by brand + price range + rating	Pass
E07	End-to-end	keyword search uses ElasticSearch	Pass
E08	End-to-end	search tolerates typos	Pass
E09	End-to-end	product detail: >= 2 variants, >= 3 images, >= 5 description lines	Pass
E10	End-to-end	guests can comment	Pass
E11	End-to-end	guests cannot rate	Pass
E12	End-to-end	customer can sign in	Pass
E13	End-to-end	signed-in customer can rate	Pass

ID	Type	Test case	Result
E14	End-to-end	customer has several delivery addresses with a default	Pass
E15	End-to-end	admin can sign in	Pass
E16	End-to-end	customer cannot open admin API	Pass
E17	End-to-end	discount code must be 5 characters	Pass
E18	End-to-end	usage limit is capped at 10	Pass
E19	End-to-end	admin creates a code	Pass
E20	End-to-end	quote shows the discount effect before paying	Pass
E21	End-to-end	quote applies loyalty points	Pass
E22	End-to-end	customer checkout succeeds	Pass
E23	End-to-end	order totals match the quote	Pass
E24	End-to-end	loyalty: earned = 10% of total in points	Pass
E25	End-to-end	points spent and earned are applied immediately	Pass
E26	End-to-end	code is rejected after reaching its usage limit	Pass
E27	End-to-end	declined card does not create an order	Pass
E28	End-to-end	order history lists the new order first	Pass
E29	End-to-end	order detail has status history	Pass
E30	End-to-end	another user cannot read the order	Pass
E31	End-to-end	admin order list (today) is newest first	Pass
E32	End-to-end	invalid status transition is rejected	Pass
E33	End-to-end	admin moves order to shipping	Pass
E34	End-to-end	customer sees status history newest first	Pass
E35	End-to-end	guest checkout succeeds and creates an account	Pass
E36	End-to-end	auto-created account is linked to the order	Pass
E37	End-to-end	admin uploads several images at once	Pass
E38	End-to-end	uploaded image is served with its detected type	Pass
E39	End-to-end	a file that is not really an image is rejected	Pass
E40	End-to-end	more than 10 files in one upload is rejected	Pass
E41	End-to-end	a product must keep at least 1 image	Pass
E42	End-to-end	a product cannot have more than 10 images	Pass
E43	End-to-end	saving 10 images keeps their order on the product page	Pass
E44	End-to-end	on-sale filter returns discounted products, biggest discount first	Pass
E45	End-to-end	product cards carry key specifications	Pass
E46	End-to-end	compare / recently viewed: products by slugs with specifications	Pass
E47	End-to-end	original price must be higher than the selling price	Pass
E48	End-to-end	wish list needs sign-in	Pass
E49	End-to-end	customer saves and merges wish-list items	Pass
E50	End-to-end	customer removes a wish-list item	Pass
E51	End-to-end	guest tracks an order with its number and e-mail	Pass
E52	End-to-end	order lookup with a wrong e-mail is refused	Pass
E53	End-to-end	SEO: robots.txt and sitemap.xml list categories, tags and products	Pass
E54	End-to-end	assistant recommends real laptops within the budget (Vietnamese)	Pass
E55	End-to-end	assistant finds a specific model (English)	Pass
E56	End-to-end	assistant answers store-policy questions	Pass
E57	End-to-end	new comments are classified by the sentiment service (RabbitMQ)	Pass
E58	End-to-end	product page includes the sentiment summary	Pass
E59	End-to-end	order confirmation e-mail reaches the guest inbox	Pass

ID	Type	Test case	Result
E60	End-to-end	simple dashboard has users, orders, revenue, best sellers	Pass
E61	End-to-end	simple dashboard has actionable insights	Pass
E62	End-to-end	advanced dashboard (year)	Pass
E63	End-to-end	advanced dashboard (quarter)	Pass
E64	End-to-end	advanced dashboard (month)	Pass
E65	End-to-end	advanced dashboard (week)	Pass
E66	End-to-end	advanced dashboard (custom range)	Pass

APPENDIX C. REQUIREMENTS TRACEABILITY AND DEMO ACCOUNTS

Table C.1 maps every item of the grading rubric to its implementation and the evidence that verifies it.

Table C.1. *Rubric items, implementation and verification*

Rubric item	Implementation	Verified by
1 Social media authentication	Google OAuth 2.0 (routes/auth.js)	Code review; needs deployer credentials (Table 8.1 #1)
2-3 Profile, change password	routes/profile.js, /auth/change-password	Browser walkthrough (Figure A.24)
4 Password recovery	/auth/forgot-password, /auth/set-password, mailer	Mailpit inbox; Figure A.23
5 Multiple delivery addresses	/profile/addresses	E14; Figure A.18
6-7 Purchase history and details	GET /orders, /orders/:number	E28-E30, E34; Figure A.25, A.26
8 Landing page	Home page sections	E02; Figure A.1, A.2
9-10 Catalogue by category, pagination everywhere	GET /products, PagedProducts component	E03; Figure A.2, A.3, A.4
11-12 Product details (5+ line overview) and variants	GET /products/:slug	E09; Figure A.8, A.10
14-16 Search, filtering, ordering	search service, lib/catalog.js	E04-E08, E44
17-18 Display and update cart	lib/cart.jsx, POST /cart/quote, mini-cart	E20, E21; Figure A.13, A.14
19 Checkout (incl. guest)	POST /orders	E22, E23, E27, E35, E36
20 Discount codes	routes/orders.js, admin/discounts.js	E17-E20, E26; U05, U06
21 E-mail after order	RabbitMQ email.send → mailer → Mailpit / Gmail	E59; U16, U17; Figure A.47
22-23 Comments, ratings, real-time update	routes/reviews.js, realtime service	E10, E11, E13; Figure A.12
24 Loyalty programme	lib/pricing.js, checkout	E21, E24, E25; U04, U09
25 Product management (incl. 1-10 images)	admin/products.js, lib/images.js, ImageManager	E37-E43, E47; U01-U03; Figure A.31-A.33
26 User management	admin/users.js	E36; Figure A.42
27-28 Simple and advanced dashboards	admin/dashboard.js	E60-E66; Figure A.36-A.39
29-30 Order list, details, status	admin/orders.js	E31-E33; Figure A.40, A.41
31 Discount management	admin/discounts.js	E17-E19; Figure A.43
34 Responsive	Bootstrap grid and breakpoints	Figure A.48-A.51
35 Horizontal scaling	Nginx + stateless replicas	E01; Table 6.1
Bonus: CI/CD	.github/workflows/ci.yml	Local runs of the same commands; GitHub screenshot by the team
Bonus: microservices	api, realtime, mailer, search, assistant	Section 6.2; Figure A.52
Bonus: AI	assistant service	E54-E58; U19-U23; Figure A.44, A.45
Bonus: ElasticSearch	search service	E07, E08; Figure A.7

Table C.2. *Demo accounts and test data*

Item	Value
Administrator	admin@techhaven.local / Admin@123
Customer with history	customer@techhaven.local / Customer@123
Discount codes with uses left	NEW26, SV100, GAME7, TECH1, VIP10, WELC5
Test cards	4242 4242 4242 4242 (approved), 4000 0000 0000 0002 (declined); any future expiry and CVC
E-mail inbox	http://localhost:8025 (Mailpit)
RabbitMQ management	http://localhost:15672 - techhaven / techhaven

REFERENCES

[1] OpenJS Foundation, "Node.js Documentation," 2024. [Online]. Available: https://nodejs.org/docs/latest/api/

[2] Express, "Express 5.x API Reference," 2024. [Online]. Available: https://expressjs.com/en/5x/api.html

[3] Meta Platforms, "React Documentation," 2024. [Online]. Available: https://react.dev/

[4] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/16/

[5] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," IETF RFC 6749, 2012.

[6] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015.

[7] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, 2011.

[8] Socket.IO, "Socket.IO v4 Documentation," 2024. [Online]. Available: https://socket.io/docs/v4/

[9] RabbitMQ, "AMQP 0-9-1 Model Explained," 2024. [Online]. Available: https://www.rabbitmq.com/tutorials/amqp-concepts

[10] Elastic, "Elasticsearch Guide 8.15," 2024. [Online]. Available: https://www.elastic.co/guide/en/elasticsearch/reference/8.15/

[11] Docker Inc., "Docker Compose Documentation," 2024. [Online]. Available: https://docs.docker.com/compose/

[12] NGINX, "HTTP Load Balancing," 2024. [Online]. Available: https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/

[13] GitHub, "GitHub Actions Documentation," 2024. [Online]. Available: https://docs.github.com/en/actions

[14] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/

[15] OWASP Foundation, "File Upload Cheat Sheet," OWASP Cheat Sheet Series, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[16] F. Å. Nielsen, "A new ANEW: Evaluation of a word list for sentiment analysis in microblogs," in Proc. ESWC2011 Workshop on Making Sense of Microposts, 2011, pp. 93-98.

[17] Google, "Gemini API Documentation," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs

[18] Schema.org, "Product," 2024. [Online]. Available: https://schema.org/Product

[19] Sitemaps.org, "Sitemaps XML format," 2024. [Online]. Available: https://www.sitemaps.org/protocol.html

[20] Axllent, "Mailpit Documentation," 2024. [Online]. Available: https://mailpit.axllent.org/docs/

[21] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, 1999, pp. 81-91.

[22] S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[23] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.